Groovy HTTP Client Documentation

Will Woodman

Table of Contents

Preface . . . 1
Introduction . . . 2
1. Overview . . . 3
1.1. What Makes This Client Different? . . . 3
1.2. Key Features . . . 3
1.2.1. Client Features . . . 3
1.2.2. Testing Features . . . 4
1.3. Requirements . . . 4
1.4. Quick Start . . . 4
1.5. Who Should Use This Library? . . . 5
1.6. Design Philosophy . . . 5
1.7. What's in This Guide . . . 5
1.8. Getting Help . . . 6
1.9. License . . . 6
1.10. Next Steps . . . 6
2. Architecture & Design Patterns . . . 7
2.1. Virtual Threads Foundation . . . 7
2.2. Key Design Patterns . . . 7
2.2.1. Builder Pattern . . . 7
2.2.2. Circuit Breaker Pattern . . . 7
2.2.3. Interceptor Chain . . . 8
Security Configuration . . . 9
3. Overview . . . 10
3.1. Design Goals . . . 10
4. Configuration Profiles . . . 11
4.1. Production Profile . . . 11
4.2. Staging Profile . . . 11
4.3. Testing Profile . . . 12
4.4. Trust-All Profile . . . 12
5. Custom Configuration . . . 14
5.1. Available Properties . . . 14
6. TLS Configuration . . . 16
6.1. Protocol Selection . . . 16
6.2. Cipher Suite Control . . . 16
6.3. Hostname Verification . . . 16
7. SSRF Protection . . . 18
7.1. Host Allow-Listing . . . 18
7.2. Absolute URL Control . . . 18

8. Cookie Policies . . . 20
9. Timeout Configuration . . . 21
9.1. Connection Timeout . . . 21
9.2. Request Timeout . . . 21
10. Retry Policy Configuration . . . 22
10.1. Basic Retry Setup . . . 22
10.2. Retry Configuration Options . . . 22
10.3. Retry Strategies . . . 22
10.4. Per-Request Retry Override . . . 23
11. Circuit Breaker Configuration . . . 24
11.1. Basic Configuration . . . 24
11.2. Circuit Breaker States . . . 24
11.3. Tuning Guidance . . . 24
12. Response Size Limits . . . 26
13. Observability . . . 27
13.1. Logging . . . 27
13.2. Metrics . . . 27
14. Best Practices . . . 28
14.1. Environment-Based Configuration . . . 28
14.2. Configuration Validation . . . 28
14.3. Security Checklist . . . 29
15. Next Steps . . . 30
URL Builder . . . 31
16. Overview . . . 32
16.1. Why Use URLBuilder? . . . 32
17. Basic Usage . . . 33
17.1. Simple URL Construction . . . 33
17.2. With Port . . . 33
17.3. With Query Parameters . . . 33
18. Query Parameters . . . 34
18.1. Single Values . . . 34
18.2. Multiple Values (Arrays) . . . 34
18.3. Bulk Parameters . . . 34
19. Path Handling . . . 36
19.1. Simple Paths . . . 36
19.2. Path With Leading Slash . . . 36
19.3. Empty Paths . . . 36
20. Closure-Based Configuration . . . 37
20.1. Mixed Usage . . . 37
21. Advanced Scenarios . . . 38
21.1. Building URLs Programmatically . . . 38

21.2. Conditional Parameters . . . 38
21.3. URL Validation . . . 39
22. Integration with GroovyHttpClient . . . 40
22.1. Using with Absolute URLs . . . 40
23. Character Encoding . . . 42
23.1. Special Characters . . . 42
24. Error Handling . . . 43
24.1. Missing Host . . . 43
24.2. Invalid URI Syntax . . . 43
25. Common Patterns . . . 44
25.1. Pagination URLs . . . 44
25.2. Filter URLs . . . 44
25.3. API Version URLs . . . 45
26. Testing with URLBuilder . . . 46
27. toString() and Build() . . . 48
28. Performance Considerations . . . 49
29. API Reference . . . 50
29.1. Constructor Options . . . 50
29.2. Fluent Methods . . . 50
29.3. Default Values . . . 50
30. Comparison with Alternatives . . . 51
30.1. vs. String Concatenation . . . 51
30.2. vs. URIBuilder (Apache HttpClient) . . . 51
31. Best Practices . . . 52
31.1. Validate Required Parameters . . . 52
31.2. Use Factory Methods . . . 52
31.3. Document URL Structure . . . 52
32. Next Steps . . . 54
GroovyHttpClient Deep Dive . . . 55
33. Overview . . . 56
33.1. Architecture . . . 56
34. Creating a Client . . . 57
34.1. Simple Construction . . . 57
34.2. With SecurityConfig . . . 57
34.3. Using Builder . . . 57
35. HTTP Methods . . . 58
35.1. GET Requests . . . 58
35.2. POST Requests . . . 58
35.3. PUT Requests . . . 59
35.4. PATCH Requests . . . 59
35.5. DELETE Requests . . . 59

35.6. HEAD Requests . . . 59
35.7. OPTIONS Requests . . . 59
36. Request Configuration DSL . . . 61
36.1. Query Parameters . . . 61
36.2. Headers . . . 62
36.3. Cookies . . . 62
36.4. Form Data . . . 63
36.5. JSON Bodies . . . 63
36.6. Custom Request Timeout . . . 64
37. Response Handling . . . 66
37.1. Response Object . . . 66
37.2. Status Code Checks . . . 66
37.3. Header Access . . . 66
37.4. JSON Responses . . . 67
37.5. XML Responses . . . 68
37.6. Binary Responses . . . 68
38. Cookie Management . . . 69
38.1. Automatic Cookies . . . 69
38.2. Adding Cookies . . . 69
38.3. Retrieving Cookies . . . 69
38.4. Removing Cookies . . . 70
38.5. Cookie Policies . . . 70
39. Default Headers . . . 71
39.1. Setting Default Headers . . . 71
39.2. Adding to Default Headers . . . 71
39.3. Clearing Default Headers . . . 71
39.4. Viewing Default Headers . . . 71
40. Downloading Files . . . 73
40.1. Binary Downloads . . . 73
40.2. With Custom Timeout . . . 73
40.3. Streaming Large Files . . . 73
41. Error Handling . . . 74
41.1. HTTP Error Responses . . . 74
41.2. Network Errors . . . 74
41.3. Circuit Breaker Errors . . . 74
41.4. Content Parse Errors . . . 75
42. Retry Policies . . . 76
42.1. Per-Client Retry Policy . . . 76
42.2. Per-Request Retry Policy . . . 76
42.3. Retry Conditions . . . 76
42.4. Backoff Strategies . . . 77

43. Request Interceptors . . . 79
43.1. Adding Interceptors . . . 79
43.2. Common Use Cases . . . 79
43.3. Interceptor Context . . . 81
44. Lifecycle Management . . . 82
44.1. Manual Cleanup . . . 82
44.2. Try-With-Resources . . . 82
44.3. Long-Lived Clients . . . 82
45. Next Steps . . . 84
Multipart Uploads . . . 85
46. Overview . . . 86
46.1. Key Features . . . 86
47. Understanding Multipart Requests . . . 87
48. DSL-Based Multipart . . . 88
48.1. Basic DSL Upload . . . 88
48.2. Text Parts . . . 88
48.3. Binary Parts . . . 89
48.4. JSON Parts . . . 89
48.5. Multiple Files . . . 90
48.6. Adding Request Headers . . . 90
48.7. Asynchronous DSL Upload . . . 91
49. Static Builder API . . . 92
49.1. Text Parts . . . 92
49.2. JSON Parts . . . 92
49.3. Binary Parts . . . 92
49.4. Mixed Content . . . 92
49.5. With Request Configuration . . . 93
50. Advanced Patterns . . . 94
50.1. Large File Streaming . . . 94
50.2. Progress Tracking . . . 94
50.3. Dynamic Part Generation . . . 95
50.4. Conditional Part Inclusion . . . 95
51. Content Type Reference . . . 97
52. Error Handling . . . 98
52.1. Upload Failures . . . 98
52.2. Validation Errors . . . 98
52.3. Retry Configuration . . . 99
53. Testing Multipart Uploads . . . 100
53.1. With HttpClientTestDsl . . . 100
53.2. With MockHttpServer . . . 100
54. Best Practices . . . 102

54.1. Choose the Right API . . . 102
54.2. Always Set Content-Type . . . 102
54.3. Use Meaningful Filenames . . . 102
54.4. Handle Large Files Appropriately . . . 103
54.5. Validate Before Upload . . . 103
55. Troubleshooting . . . 104
55.1. "Multipart parts are required" Error . . . 104
55.2. Missing Filename . . . 104
55.3. Content-Type Issues . . . 104
55.4. Boundary Collisions . . . 104
56. Performance Considerations . . . 106
57. Next Steps . . . 107
Circuit Breaker . . . 108
58. Overview . . . 109
58.1. Key Features . . . 109
59. Circuit Breaker States . . . 110
59.1. CLOSED State . . . 110
59.2. OPEN State . . . 110
59.3. HALF_OPEN State . . . 111
60. Configuration . . . 112
60.1. Via SecurityConfig . . . 112
60.2. Via Builder . . . 112
60.3. Configuration Parameters . . . 112
61. Failure Classification . . . 113
61.1. Counted Failures (Circuit Breaker Opens) . . . 113
61.2. Ignored Failures (Circuit Stays Closed) . . . 113
62. Usage Patterns . . . 114
62.1. Automatic Operation . . . 114
62.2. Graceful Degradation . . . 114
62.3. Health Checks . . . 114
62.4. Multiple Backends . . . 115
63. Integration with Retries . . . 116
64. Advanced Patterns . . . 117
64.1. Manual State Inspection . . . 117
64.2. Metrics Collection . . . 117
64.3. Circuit Breaker Per Endpoint . . . 118
65. Testing Circuit Breakers . . . 120
65.1. With Spock . . . 120
65.2. Simulating Failures . . . 121
66. Best Practices . . . 123
66.1. Choose Appropriate Thresholds . . . 123

66.2. Scale Reset Timeout with Recovery Time . . . 123
66.3. Combine with Retries Carefully . . . 123
66.4. Always Handle CircuitOpenException . . . 124
66.5. Monitor Circuit Breaker State . . . 124
66.6. Don’t Disable for Testing . . . 125
67. Troubleshooting . . . 126
67.1. Circuit Opens Too Quickly . . . 126
67.2. Circuit Never Opens . . . 126
67.3. Circuit Stays Open Forever . . . 126
67.4. Probe Request Fails . . . 127
68. Performance Impact . . . 128
69. Next Steps . . . 129
Testing with MockHttpServer . . . 130
70. Overview . . . 131
70.1. Key Features . . . 131
70.2. When to Use MockHttpServer . . . 131
71. Quick Start . . . 132
71.1. Basic Server Setup . . . 132
71.2. With Groovy Try-With-Resources . . . 132
72. Request Validation . . . 133
72.1. Method Validation . . . 133
72.2. Header Validation . . . 133
72.3. Cookie Validation . . . 133
72.4. Query Parameter Validation . . . 134
73. Response Configuration . . . 135
73.1. Status Codes . . . 135
73.2. Response Bodies . . . 135
73.3. Response Headers . . . 136
73.4. Response Cookies . . . 136
74. Simulating Latency . . . 137
75. Error Simulation . . . 138
75.1. Client Errors (4xx) . . . 138
75.2. Server Errors (5xx) . . . 138
75.3. Testing Retry Logic . . . 138
76. Multipart Request Testing . . . 140
77. Spock Integration . . . 141
77.1. Setup/Cleanup Patterns . . . 142
78. JUnit 5 Integration . . . 144
79. Advanced Patterns . . . 145
79.1. Dynamic Response Generation . . . 145
79.2. Multiple Endpoints . . . 145

79.3. Testing Redirect Handling . . . 145
79.4. Cookie Flow Testing . . . 146
80. Best Practices . . . 147
80.1. Use Descriptive Endpoints . . . 147
80.2. Organize Tests by Scenario . . . 147
80.3. Reset State Between Tests . . . 147
80.4. Use Realistic Response Data . . . 148
80.5. Test Error Scenarios . . . 148
81. Limitations . . . 149
82. Troubleshooting . . . 150
82.1. Port Already in Use . . . 150
82.2. Headers Not Matching . . . 150
82.3. Cookies Not Matching . . . 150
82.4. Server Not Starting . . . 150
83. Next Steps . . . 151
Testing DSL (HttpClientTestDsl) . . . 152
84. Overview . . . 153
84.1. Key Features . . . 153
84.2. When to Use HttpClientTestDsl . . . 153
85. Quick Start . . . 154
85.1. Basic Server Setup . . . 154
85.2. With Try-With-Resources . . . 155
86. Defining Mock Endpoints . . . 156
86.1. Basic Endpoint . . . 156
86.2. Multiple Endpoints . . . 156
86.3. Shorthand Methods . . . 157
87. Response Configuration . . . 158
87.1. Status Codes . . . 158
87.2. Response Bodies . . . 158
87.3. Response Headers . . . 159
87.4. Custom Content Types . . . 160
88. Request Recording and Verification . . . 161
88.1. Basic Verification . . . 161
88.2. Method Verification . . . 161
88.3. Header Verification . . . 161
88.4. Body Verification . . . 162
88.5. Query Parameter Verification . . . 163
88.6. Accessing Recorded Requests . . . 163
88.7. RecordedRequest Properties . . . 164
89. Spock Integration . . . 165
89.1. Setup/Cleanup Pattern . . . 165

89.2. Shared Server Pattern . . . 166
89.3. Complex Verification Example . . . 166
90. JUnit 5 Integration . . . 168
91. Advanced Patterns . . . 170
91.1. Testing Async Operations . . . 170
91.2. Testing Request Order . . . 171
91.3. Testing Error Recovery . . . 171
91.4. Testing Multipart Uploads . . . 172
91.5. Testing Circuit Breaker Behavior . . . 172
92. Best Practices . . . 174
92.1. Reset State Between Tests . . . 174
92.2. Use Realistic Response Data . . . 174
93. Limitations . . . 177
94. Troubleshooting . . . 178
95. Performance Tips . . . 180
96. Next Steps . . . 181
HttpClientTestTrait . . . 182
97. Overview . . . 183
97.1. Key Benefits . . . 183
98. Quick Start . . . 184
98.1. Basic Usage . . . 184
98.2. What the Trait Provides . . . 184
99. Trait Methods . . . 185
99.1. Server Access . . . 185
99.2. Client Creation . . . 185
99.3. Endpoint Mocking . . . 185
99.4. Request Verification . . . 186
99.5. Server Management . . . 186
100. Complete Example . . . 188
101. Advanced Usage . . . 192
101.1. Multiple Endpoints . . . 192
101.2. Error Scenarios . . . 192
101.3. Testing with Cookies . . . 193
102. Data-Driven Tests . . . 195
103. Testing Patterns . . . 197
103.1. Testing Paginated APIs . . . 197
103.2. Testing Error Recovery . . . 197
103.3. Testing Rate Limiting . . . 198
104. Trait Implementation Details . . . 199
105. Comparison: Direct vs Trait . . . 200
106. Best Practices . . . 201

106.1. Reset Between Tests . . . 201
106.2. Use Helper Methods . . . 201
106.3. Document Test Intent . . . 202
107. Limitations . . . 203
108. Troubleshooting . . . 204
108.1. Tests Interfering with Each Other . . . 204
108.2. "Server not started" Error . . . 204
108.3. Port Conflicts . . . 204
109. Next Steps . . . 205
Real-World Examples . . . 206
110. Overview . . . 207
111. Basic HTTP Operations . . . 208
111.1. Simple GET Request . . . 208
111.2. POST with JSON Body . . . 208
111.3. PUT Request to Update Resource . . . 209
111.4. DELETE Request . . . 209
112. Authentication Patterns . . . 210
112.1. Bearer Token Authentication . . . 210
112.2. API Key Authentication . . . 210
112.3. Basic Authentication . . . 210
112.4. OAuth 2.0 Flow . . . 211
113. Query Parameters . . . 212
113.1. Simple Query Parameters . . . 212
113.2. Array Query Parameters . . . 212
113.3. Pagination . . . 212
114. File Operations . . . 214
114.1. Upload Single File . . . 214
114.2. Upload Multiple Files . . . 214
114.3. Download File . . . 215
114.4. Download with Progress Tracking . . . 215
115. Error Handling . . . 217
115.1. Handling HTTP Errors . . . 217
115.2. Retry with Exponential Backoff . . . 217
115.3. Circuit Breaker Pattern . . . 218
115.4. Graceful Degradation . . . 218
116. Async Operations . . . 220
116.1. Concurrent Requests . . . 220
116.2. Async with Error Handling . . . 220
116.3. Parallel Processing . . . 221
117. Testing Examples . . . 222
117.1. Testing with HttpClientTestDsl . . . 222

117.2. Testing Multipart Uploads . . . 223
118. Production Patterns . . . 225
118.1. Rate Limiting . . . 225
118.2. Configuration Management . . . 227
119. Additional Examples in Test Suite . . . 229
119.1. Client Tests . . . 229
119.2. Testing Framework Tests . . . 229
119.3. Running the Tests . . . 229
119.4. Test Report . . . 229
120. Next Steps . . . 231
Appendix A: API Reference . . . 232
Appendix B: Troubleshooting . . . 233

Preface

A modern, production-ready HTTP client built on Java 25 virtual threads with comprehensive Groovy DSL support, and testing framework support.

Introduction

Java Virtual Threads *HTTP/2* *Groovy 4.0+*

Chapter 1. Overview

The **Groovy HTTP Client** is a modern, production-grade HTTP client library for Groovy and Java applications. Built on Java's `HttpClient` API and leveraging virtual threads from Project Loom, it provides a powerful, fluent DSL for making HTTP requests with enterprise-grade features including circuit breakers, retry policies, security controls, and comprehensive testing support.

> A Groovy-idiomatic HTTP client that doesn't compromise on performance, security, or testability.

1.1. What Makes This Client Different?

Traditional HTTP clients often require choosing between ease of use and performance, or between rich features and simplicity. The Groovy HTTP Client eliminates these trade-offs:

Groovy DSL	Natural, expressive syntax that feels like writing configuration, not code
Virtual Thread Architecture	Millions of concurrent requests with minimal resource overhead
Security by Default	Comprehensive security controls with environment-aware profiles
Testing First	Purpose-built testing framework with mock server and fluent assertions
Production Ready	Circuit breakers, retry policies, metrics, and structured logging
Zero Magic	Wraps Java's standard `HttpClient` — no reflection, no bytecode manipulation

1.2. Key Features

1.2.1. Client Features

- **Fluent Groovy DSL** for request configuration, headers, and parameters
- **Automatic cookie management** with configurable policies
- **Multipart form uploads** via DSL or programmatic API
- **Circuit breaker** for resilience (CLOSED → OPEN → HALF_OPEN states)
- **Retry policies** with exponential backoff and jitter
- **Virtual thread executor** for ultra-lightweight concurrency
- **Request/response interceptors** for logging, metrics, and custom processing

- **Comprehensive security model** via `SecurityConfig` profiles
- **SSRF protection** with host allow-listing
- **HTTP/2 support** with automatic protocol negotiation

1.2.2. Testing Features

- **MockHttpServer** — Lightweight, dependency-free test server
- **HttpClientTestDsl** — Fluent DSL for endpoint mocking and verification
- **HttpClientTestTrait** — Spock trait for seamless test integration
- **Request recording** with rich assertion capabilities
- **Multipart validation** for file upload testing
- **Header and cookie verification** built-in

1.3. Requirements

Requirement	Version
Java	21 or higher (for virtual threads)
Groovy	5.0 or higher (tested with 5.0.4)
Build Tool	Gradle 9+ or Maven 3.8+



Virtual threads are a preview feature in Java 19-20 and production-ready in Java 21+.

1.4. Quick Start

Add the dependency to your project:

build.gradle

```
dependencies {
    implementation 'org.softwood:groovy-HttpClient:1.5.0'
}
```

Create a client and make your first request:

```
import org.softwood.http.GroovyHttpClient

// Simple client with testing defaults
def client = new GroovyHttpClient("https://api.example.com")

// Make a GET request
def response = client.getSync("/users/123")
```

```
println "Status: ${response.statusCode}"
println "Body: ${response.body}"

// Access parsed JSON
def user = response.json
println "User name: ${user.name}"
```

1.5. Who Should Use This Library?

This library is designed for:

Groovy Developers	Who want idiomatic, expressive HTTP client code
API Integration Projects	Needing robust retry logic and circuit breakers
Microservices	Requiring high-concurrency HTTP communication
Testing Scenarios	Where mock servers and request verification are critical
Security-Conscious Teams	Needing fine-grained control over TLS, timeouts, and SSRF protection
Java Developers	Looking for a modern alternative to Apache HttpClient or OkHttp

1.6. Design Philosophy

The library follows these core principles:

1. **Groovy-First, Java-Compatible**: Leverage Groovy's strengths while remaining usable from Java
2. **Wrap, Don't Replace**: Build on Java's standard `HttpClient` rather than reinventing HTTP
3. **Security by Configuration**: Make secure defaults easy, insecure options explicit
4. **Test-Driven Design**: Testing capabilities are first-class, not afterthoughts
5. **Performance Through Simplicity**: Virtual threads eliminate need for complex async patterns
6. **Fail Fast**: Validation happens early, errors are clear and actionable

1.7. What's in This Guide

This guide is organized into three main sections:

Core Concepts (Chapters 2-7)

Covers the architecture, security model, core client features, and multipart uploads. Read these chapters to understand how to configure and use the client effectively.

Testing Framework (Chapters 8-10)

Describes the `MockHttpServer`, `HttpClientTestDsl`, and `HttpClientTestTrait` for comprehensive

HTTP testing. Essential reading for writing robust tests.

Examples & Reference (Chapters 11-12)

Provides real-world examples, troubleshooting guidance, and API reference material.

1.8. Getting Help

Documentation	This guide and inline Javadoc/GroovyDoc
Source Code	github.com/softwood/groovy-HttpClient
Issue Tracking	GitHub Issues
Email Support	will@softwood.org



The source code is extensively commented and can serve as additional documentation.

1.9. License

This library is licensed under the Apache License 2.0. See the `LICENSE` file for details.

```
Copyright 2025 Softwood Consulting Ltd

Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
```

1.10. Next Steps

Continue reading to learn about:

- Architecture & Design Patterns — Understanding the virtual thread foundation
- Security Configuration — Environment profiles and security controls
- URL Builder — Constructing complex URLs with query parameters
- GroovyHttpClient Deep Dive — Comprehensive client usage guide

Let's dive in!

Chapter 2. Architecture & Design Patterns

2.1. Virtual Threads Foundation

The `GroovyHttpClient` is built on **Java 21+ virtual threads** (Project Loom), providing:

Lightweight concurrency	Millions of concurrent requests with minimal overhead
Blocking-friendly	Natural blocking APIs without thread pool exhaustion
Structured concurrency	Clean resource management

```
ThreadFactory tf = Thread.ofVirtual()
    .name("http-client-", counter)
    .factory()

executor = Executors.newThreadPerTaskExecutor(tf)
```



Virtual threads are perfect for I/O-bound operations like HTTP requests.

2.2. Key Design Patterns

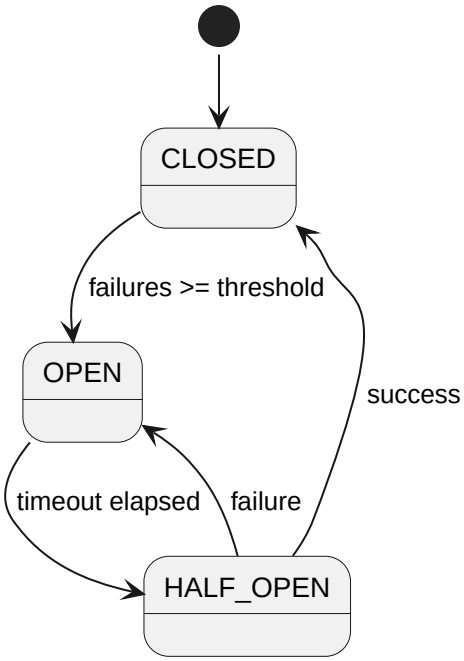
2.2.1. Builder Pattern

The `SecurityConfig` and `URLBuilder` use fluent builders:

```
def config = SecurityConfig.builder("https://api.example.com")
    .connectTimeout(Duration.ofSeconds(5))
    .allowRedirects(true)
    .failureThreshold(3)
    .build()
```

2.2.2. Circuit Breaker Pattern

Implements the circuit breaker pattern with three states:



See Circuit Breaker for details.

2.2.3. Interceptor Chain

Supports middleware-style request/response interceptors:

```
client.addInterceptor { ctx, next ->
    println "Request: ${ctx.method} ${ctx.uri}"
    next.call(ctx).thenApply { resp ->
        println "Response: ${resp.statusCode}"
        resp
    }
}
```



Interceptors execute in LIFO order (last added runs first).

Security Configuration

Chapter 3. Overview

The `SecurityConfig` class is the cornerstone of the Groovy HTTP Client's security model. It provides a declarative, environment-aware approach to configuring TLS policies, timeout controls, SSRF protection, cookie handling, and resilience patterns.



Security is not an add-on in this library—it's baked into the foundation. Every `GroovyHttpClient` instance requires a `SecurityConfig`, ensuring security decisions are explicit and auditable.

3.1. Design Goals

Environment Awareness	Different security needs for production vs. testing
Fail-Safe Defaults	Secure by default, insecure by opt-in
Configuration as Code	Security policies version-controlled alongside application code
Zero Runtime Magic	No classpath scanning, no reflection-based configuration loading
Immutability	Configuration cannot be changed after creation, preventing drift

Chapter 4. Configuration Profiles

`SecurityConfig` provides four built-in profiles optimized for different environments.

4.1. Production Profile

The production profile enforces the strictest security controls:

```
def config = SecurityConfig.production("https://api.example.com")

def client = new GroovyHttpClient(config)
```

Table 1. Production Profile Settings

Property	Value	Rationale
`insecureAllowed`	`false`	Only verified TLS connections
`enforceHostnameVerification`	`true`	Prevent MITM attacks
`allowedTlsProtocols`	`["TLSv1.3"]`	Latest secure protocol only
`allowAbsoluteUrls`	`false`	SSRF protection
`cookiePolicy`	`ACCEPT_ORIGINAL_SERVER`	Prevent cookie injection
`maxResponseBytes`	`10,000,000` (10 MB)	DoS protection
`connectTimeout`	`5 seconds`	Fail fast on network issues
`requestTimeout`	`20 seconds`	Prevent hung connections
`failureThreshold`	`5`	Circuit breaker sensitivity
`retryPolicy`	Enabled (3 attempts)	Automatic recovery from transient failures



Production profiles disable all "trust-all" TLS modes. Self-signed certificates will fail unless properly imported into the JVM truststore.

4.2. Staging Profile

Staging mirrors production but with enhanced observability:

```
def config = SecurityConfig.staging("https://staging.api.example.com")
```

Differences from Production

- `enableLogging = true` — Request/response logging enabled
- `requestTimeout = 30 seconds` — Allows longer debugging sessions

- Same security controls as production



Use staging profiles to validate security policies before production deployment.

4.3. Testing Profile

The testing profile relaxes security controls for unit and integration tests:

```
def config = SecurityConfig.testing("http://localhost:8080")

// Client can now accept self-signed certs, absolute URLs, etc.
def client = new GroovyHttpClient(config)
```

Table 2. Testing Profile Settings

Property	Value
`insecureAllowed`	`true` — Accepts self-signed certificates
`allowAbsoluteUrls`	`true` — Can call any host
`cookiePolicy`	`ACCEPT_ALL` — Simplified cookie handling
`retryPolicy`	Disabled by default (1 attempt)
`enableLogging`	`true` — Full request/response visibility
`enableMetrics`	`false` — Reduced overhead



Never use testing profiles in production. They disable critical security controls and can expose your application to attacks.

4.4. Trust-All Profile

The most permissive profile for local development and debugging:

```
def config = SecurityConfig.trustAll("http://localhost:3000")
```

This profile disables:

- TLS certificate validation
- Hostname verification
- Host allow-listing
- Most timeout enforcement



The `trustAll()` profile should **only** be used for:

- Local development against `localhost`

> - Debugging TLS issues
> - Throwaway proof-of-concept code
>
> It must never be used in production, staging, or any network-accessible environment.

Chapter 5. Custom Configuration

All profiles can be customized using `withOverrides()`:

```
def config = SecurityConfig.production("https://api.example.com")
    .withOverrides([
        requestTimeout: Duration.ofSeconds(60),
        enableLogging: true,
        failureThreshold: 3,
        allowedHosts: ["api.example.com", "cdn.example.com"] as Set
    ])
```

5.1. Available Properties

Table 3. Core Settings

Property	Type	Description
`baseUrl`	`String`	Root URL for all requests
`connectTimeout`	`Duration`	TCP handshake timeout
`requestTimeout`	`Duration`	Per-request timeout (entire request/response cycle)
`maxResponseBytes`	`Long`	Maximum response body size (DoS protection)
`allowRedirects`	`Boolean`	Follow HTTP redirects automatically

Table 4. Security Settings

Property	Type	Description
`insecureAllowed`	`Boolean`	Allow self-signed certificates (trust-all mode)
`enforceHostnameVerification`	`Boolean`	Verify TLS certificate hostname matches target
`allowedTlsProtocols`	`List<String>`	Permitted TLS protocol versions
`allowedCipherSuites`	`List<String>`	Permitted cipher suites (empty = JVM defaults)
`allowAbsoluteUrls`	`Boolean`	Allow requests to absolute URLs outside `baseUrl`
`allowedHosts`	`Set<String>`	Host whitelist for SSRF protection
`cookiePolicy`	`CookiePolicy`	Cookie acceptance policy

Table 5. Resilience Settings

Property	Type	Description
`failureThreshold`	`Integer`	Failures before circuit breaker opens
`resetTimeoutMs`	`Long`	Circuit breaker reset delay (milliseconds)
`retryPolicy`	`RetryPolicy`	Retry configuration (attempts, backoff, conditions)

Table 6. Observability Settings

Property	Type	Description
`enableLogging`	`Boolean`	Log requests and responses
`enableMetrics`	`Boolean`	Collect request metrics (future feature)

Chapter 6. TLS Configuration

6.1. Protocol Selection

Specify allowed TLS versions:

```
def config = SecurityConfig.production("https://api.example.com")
    .withOverrides([
        allowedTlsProtocols: ["TLSv1.3", "TLSv1.2"]
    ])
```

Protocol	Recommendation
`TLSv1.3`	Preferred — modern, secure, performant
`TLSv1.2`	Acceptable — widely supported
`TLSv1.1`, `TLSv1.0`	Deprecated — should not be used
`SSLv3`, `SSLv2`	Insecure — never use

6.2. Cipher Suite Control

Restrict cipher suites (empty list = JVM defaults):

```
def config = SecurityConfig.production("https://api.example.com")
    .withOverrides([
        allowedCipherSuites: [
            "TLS_AES_256_GCM_SHA384",
            "TLS_AES_128_GCM_SHA256"
        ]
    ])
```



Cipher suite names are Java-specific. Use `SSLContext.getDefault().getSupportedSSLParameters().getCipherSuites()` to list available suites.

6.3. Hostname Verification

Control whether TLS certificate hostnames must match:

```
// Production: hostname verification required
def prod = SecurityConfig.production("https://api.example.com")

// Testing: hostname verification disabled
def test = SecurityConfig.testing("https://api.example.com")
```

```
    .withOverrides([
        enforceHostnameVerification: false
    ])
```



Disabling hostname verification opens you to man-in-the-middle attacks. Only disable for:

- Testing against hosts with wrong SANs
- Local development environments
- Debugging TLS issues

Never disable in production.

Chapter 7. SSRF Protection

Server-Side Request Forgery (SSRF) protection prevents attackers from making your server call arbitrary URLs.

7.1. Host Allow-Listing

Restrict which hosts the client can contact:

```
def config = SecurityConfig.production("https://api.example.com")
    .withOverrides([
        allowAbsoluteUrls: true,
        allowedHosts: [
            "api.example.com",
            "cdn.example.com",
            "auth.example.com"
        ] as Set
    ])

def client = new GroovyHttpClient(config)

// ✅ Allowed
client.getSync("https://api.example.com/data")

// ❌ Throws IllegalArgumentException
client.getSync("https://evil.com/steal-data")
```



The `baseUrl` host is **automatically** added to `allowedHosts`. You only need to list additional trusted hosts.

7.2. Absolute URL Control

Disable absolute URLs entirely for maximum protection:

```
def config = SecurityConfig.production("https://api.example.com")
    .withOverrides([
        allowAbsoluteUrls: false
    ])

def client = new GroovyHttpClient(config)

// ✅ Allowed (relative to baseUrl)
client.getSync("/users")

// ❌ Throws IllegalArgumentException
client.getSync("https://api.example.com/users")
```



For microservices communicating with a single API, set `allowAbsoluteUrls = false` to eliminate SSRF risk entirely.

Chapter 8. Cookie Policies

Configure how cookies are handled using Java's `CookiePolicy`:

```
import java.net.CookiePolicy

def config = SecurityConfig.production("https://api.example.com")
    .withOverrides([
        cookiePolicy: CookiePolicy.ACCEPT_ORIGINAL_SERVER
    ])
```

Table 7. Available Cookie Policies

Policy	Behavior
`ACCEPT_ALL`	Accept all cookies from any domain
`ACCEPT_NONE`	Reject all cookies
`ACCEPT_ORIGINAL_SERVER`	Only accept cookies from the origin server (default for production)



The cookie policy works in conjunction with the automatic `CookieManager`. See Cookie Management for usage examples.

Chapter 9. Timeout Configuration

Proper timeout configuration is critical for application stability.

9.1. Connection Timeout

Time allowed for TCP handshake:

```
def config = SecurityConfig.production("https://api.example.com")
    .withOverrides([
        connectTimeout: Duration.ofSeconds(3)
    ])
```

- **Too Low**: False failures on slow networks
- **Too High**: Slow failure detection, resource exhaustion
- **Recommended**: 3-10 seconds for production

9.2. Request Timeout

Total time for complete request/response cycle:

```
def config = SecurityConfig.production("https://api.example.com")
    .withOverrides([
        requestTimeout: Duration.ofSeconds(30)
    ])
```

- **Too Low**: Fails legitimate slow requests (large payloads, slow APIs)
- **Too High**: Hung connections, cascading failures
- **Recommended**: 20-60 seconds depending on use case



Set timeout values based on your API's P99 latency:

- Fast APIs (<100ms P99): 10-20 seconds
- Medium APIs (100-500ms P99): 20-40 seconds
- Slow APIs (>500ms P99): 40-60 seconds

Never set infinite timeouts in production.

Chapter 10. Retry Policy Configuration

Configure automatic retry behavior for transient failures.

10.1. Basic Retry Setup

```
import org.softwood.http.GroovyHttpClient.RetryPolicy

def retryPolicy = new RetryPolicy.Config().with {
    maxAttempts(3)
    backoffExponential(200, 2000, true) // initial, max, jitter
    onIoExceptions(true)
    onTimeouts(true)
    on5xx(true)
    build()
}

def config = SecurityConfig.production("https://api.example.com")
    .withOverrides([
        retryPolicy: retryPolicy
    ])
```

10.2. Retry Configuration Options

Table 8. RetryPolicy.Config Methods

Method	Description
`maxAttempts(int)`	Total attempts including original (1 = no retries)
`backoffExponential(long, long, boolean)`	Configure exponential backoff: initial delay (ms), max delay (ms), jitter enabled
`onIoExceptions(boolean)`	Retry on I/O exceptions (connection failures, etc.)
`onTimeouts(boolean)`	Retry on timeout exceptions
`on5xx(boolean)`	Retry on HTTP 5xx server errors
`onStatusCodes(Collection<Integer>)`	Retry on specific status codes

10.3. Retry Strategies

Exponential Backoff with Jitter:

```
// Initial: 100ms, doubles each attempt with random jitter
// Max delay: 5 seconds
backoffExponential(100, 5000, true)
```

Fixed Delay:

```
// Always wait 1 second between attempts
backoffExponential(1000, 1000, false)
```

No Retries:

```
RetryPolicy.none() // or maxAttempts(1)
```

10.4. Per-Request Retry Override

Override retry policy for specific requests:

```
client.post("/critical-operation", data) {
    retry {
        maxAttempts(5)
        backoffExponential(500, 10000, true)
        on5xx(true)
    }
}
```



Be cautious with retries on non-idempotent operations (POST, PATCH). Retrying can result in duplicate actions (double-charging, duplicate records, etc.).

Consider implementing idempotency tokens for non-GET requests that need retries.

Chapter 11. Circuit Breaker Configuration

Protect your application from cascading failures with circuit breakers.

11.1. Basic Configuration

```
def config = SecurityConfig.production("https://api.example.com")
    .withOverrides([
        failureThreshold: 5,        // Open after 5 failures
        resetTimeoutMs: 30_000L     // Wait 30s before half-open
    ])
```

11.2. Circuit Breaker States

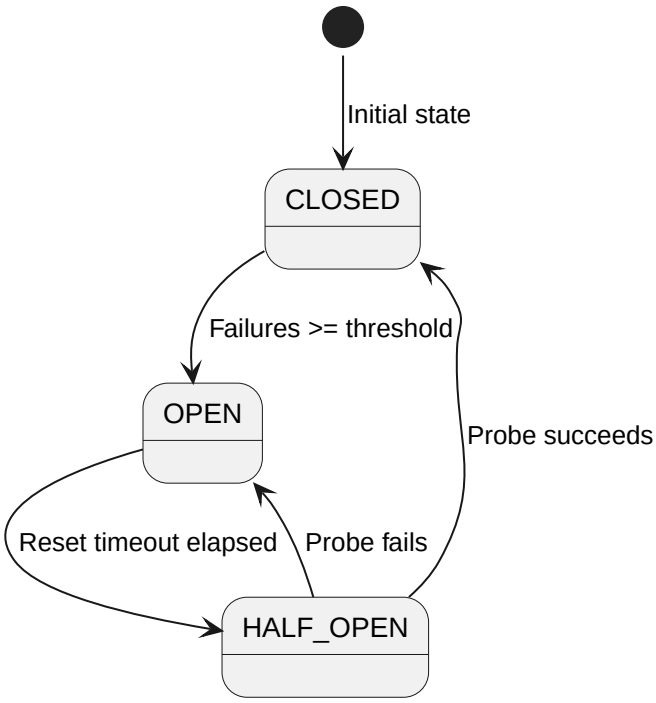


Table 9. State Descriptions

State	Behavior
CLOSED	Normal operation. All requests proceed. Failures are counted.
OPEN	Circuit tripped. All requests immediately fail. No calls to downstream service.
HALF_OPEN	Testing phase. Single probe request sent. Success → CLOSED, Failure → OPEN.

11.3. Tuning Guidance

Scenario	Configuration
High-traffic API	`failureThreshold: 10+`, `resetTimeoutMs: 60000`
Low-traffic API	`failureThreshold: 3-5`, `resetTimeoutMs: 30000`
Flaky service	`failureThreshold: 15+`, `resetTimeoutMs: 120000`
Critical path	`failureThreshold: 2-3`, `resetTimeoutMs: 10000`



Circuit breaker state is per-client instance. If you create multiple `GroovyHttpClient` instances, each has its own circuit breaker.

Chapter 12. Response Size Limits

Protect against memory exhaustion from large responses:

```
def config = SecurityConfig.production("https://api.example.com")
    .withOverrides([
        maxResponseBytes: 5_000_000L  // 5 MB limit
    ])
```

Requests exceeding this limit throw `HttpResponseException` with status 413:

```
try {
    client.getSync("/large-file")
} catch (GroovyHttpClient.HttpResponseException e) {
    if (e.statusCode == 413) {
        println "Response too large"
    }
}
```



Set `maxResponseBytes` based on your use case:

- APIs returning small JSON/XML: 1-10 MB
- File downloads: 100-500 MB or `Long.MAX_VALUE`
- Streaming APIs: Use streaming APIs, not HTTP client

Chapter 13. Observability

13.1. Logging

Enable request/response logging:

```
def config = SecurityConfig.production("https://api.example.com")
    .withOverrides([
        enableLogging: true
    ])
```

Logs include:

- Request method, URL, headers
- Response status, headers, body (truncated for large responses)
- Request timing
- Retry attempts
- Circuit breaker state changes



Logging uses SLF4J. Configure your logging framework (Logback, Log4j2) to control output.

13.2. Metrics

Metrics collection (future feature):

```
def config = SecurityConfig.production("https://api.example.com")
    .withOverrides([
        enableMetrics: true
    ])
```

Planned metrics include:

- Request count, rate
- Response time distributions
- Error rates by status code
- Circuit breaker state durations
- Retry attempt counts



Metrics support is planned for a future release. The API is included now for forward compatibility.

Chapter 14. Best Practices

14.1. Environment-Based Configuration

Use different configs per environment:

```
class AppConfig {
    static SecurityConfig httpConfig() {
        def env = System.getenv("APP_ENV") ?: "production"

        switch (env) {
            case "production":
                return SecurityConfig.production(getApiUrl())
                    .withOverrides([
                        failureThreshold: 5,
                        retryPolicy: productionRetryPolicy()
                    ])

            case "staging":
                return SecurityConfig.staging(getApiUrl())

            case "test":
                return SecurityConfig.testing("http://localhost:8080")

            default:
                throw new IllegalStateException("Unknown environment: $env")
        }
    }

    private static RetryPolicy productionRetryPolicy() {
        new RetryPolicy.Config().with {
            maxAttempts(3)
            backoffExponential(200, 2000, true)
            on5xx(true)
            onTimeouts(true)
            build()
        }
    }

    private static String getApiUrl() {
        System.getenv("API_URL") ?: "https://api.example.com"
    }
}
```

14.2. Configuration Validation

Validate configuration at application startup:

```
def config = SecurityConfig.production(apiUrl)

// Validate configuration makes sense
assert config.connectTimeout < config.requestTimeout :
    "Connect timeout must be less than request timeout"

assert config.failureThreshold > 0 :
    "Failure threshold must be positive"

assert !config.insecureAllowed :
    "Insecure TLS not allowed in production"
```

14.3. Security Checklist

Before deploying to production, verify:

- ☐ `insecureAllowed = false`
- ☐ `enforceHostnameVerification = true`
- ☐ `allowedTlsProtocols = ["TLSv1.3"]` or `["TLSv1.3", "TLSv1.2"]`
- ☐ `allowAbsoluteUrls` and `allowedHosts` configured appropriately
- ☐ `maxResponseBytes` set to reasonable limit
- ☐ Timeouts configured based on API characteristics
- ☐ Circuit breaker thresholds tuned for traffic patterns
- ☐ Retry policy safe for non-idempotent operations
- ☐ Configuration externalized (environment variables, config files)

Chapter 15. Next Steps

Now that you understand security configuration, learn about:

- URLBuilder — Constructing complex URLs with query parameters
- GroovyHttpClient Deep Dive — Making requests with the configured client
- Circuit Breaker Details — Understanding failure handling in depth

URL Builder

Chapter 16. Overview

The `URLBuilder` class provides a fluent, type-safe API for constructing URLs with proper encoding, query parameters, and path handling. It eliminates common pitfalls like forgetting to URL-encode parameters or incorrectly handling array values.

> URL construction should be declarative, not string concatenation soup.

16.1. Why Use URLBuilder?

Without URLBuilder (error-prone):

```
// ❌ Multiple problems:
// - No URL encoding
// - Manual string concatenation
// - Error-prone for arrays
// - Hard to read and maintain
def url = "https://api.example.com/search?q=" + query +
          "&tags=" + tags.join(",") +
          "&limit=" + limit
```

With URLBuilder (safe and clear):

```
// ✅ Proper encoding, clear intent, array-safe
def url = new URLBuilder("api.example.com")
    .protocol("https")
    .path("/search")
    .param("q", query)
    .param("tags", tags)      // Array handled automatically
    .param("limit", limit)
    .build()
```

Chapter 17. Basic Usage

17.1. Simple URL Construction

```
import org.softwood.url.URLBuilder

def url = new URLBuilder("api.example.com")
    .protocol("https")
    .path("/users")
    .build()

// Result: https://api.example.com/users
```

17.2. With Port

```
def url = new URLBuilder("localhost")
    .protocol("http")
    .port(8080)
    .path("/api/v1/status")
    .build()

// Result: http://localhost:8080/api/v1/status
```

17.3. With Query Parameters

```
def url = new URLBuilder("api.example.com")
    .protocol("https")
    .path("/search")
    .param("q", "groovy http client")
    .param("limit", 10)
    .param("offset", 0)
    .build()

// Result: https://api.example.com/search?q=groovy+http+client&limit=10&offset=0
```



All parameter values are automatically URL-encoded. Spaces become `+`, special characters are percent-encoded.

Chapter 18. Query Parameters

18.1. Single Values

Add parameters one at a time:

```
def builder = new URLBuilder("api.example.com")
    .param("userId", 123)
    .param("format", "json")
    .param("pretty", true)
```

18.2. Multiple Values (Arrays)

URLBuilder automatically handles collections and ranges:

```
def tags = ["groovy", "http", "client"]

def url = new URLBuilder("api.example.com")
    .path("/articles")
    .param("tags", tags)        // Collection
    .build()

// Result: https://api.example.com/articles?tags=groovy&tags=http&tags=client
```

Works with:

- `List` — `param("ids", [1, 2, 3])`
- `Set` — `param("tags", ["a", "b"] as Set)`
- `Array` — `param("vals", new String[]{"x", "y"})`
- `Range` — `param("ids", 1..10)`

18.3. Bulk Parameters

Add multiple parameters at once:

```
def params = [
    q: "search term",
    limit: 50,
    offset: 100,
    sort: "date",
    order: "desc"
]

def url = new URLBuilder("api.example.com")
```

```
    .path("/search")
    .params(params)
    .build()
```



`params()` can be called multiple times. Parameters accumulate.

Chapter 19. Path Handling

19.1. Simple Paths

```
def url = new URLBuilder("api.example.com")
    .path("/users/123/profile")
    .build()
```

19.2. Path With Leading Slash

Leading slashes are handled automatically:

```
// With slash
new URLBuilder("api.example.com")
    .path("/api/users")
    .build()
// → https://api.example.com/api/users

// Without slash
new URLBuilder("api.example.com")
    .path("api/users")
    .build()
// → https://api.example.com/api/users
```

19.3. Empty Paths

Omitting the path results in just the host:

```
def url = new URLBuilder("api.example.com")
    .build()
// → https://api.example.com
```



URLBuilder does not perform path segment encoding. If your paths contain special characters, you must encode them yourself:

```
def fileName = "my document.pdf"
def encoded = URLEncoder.encode(fileName, "UTF-8")

def url = new URLBuilder("cdn.example.com")
    .path("/files/$encoded")
    .build()
```

Chapter 20. Closure-Based Configuration

For complex URLs, use closure-based configuration:

```
def url = new URLBuilder {
    protocol "https"
    host "api.example.com"
    port 8443
    path "/search"

    param "q", "groovy"
    param "tags", ["http", "client"]
    param "limit", 10
}.build()
```

This is equivalent to the fluent API but reads more like configuration.

20.1. Mixed Usage

You can combine constructor arguments with closures:

```
def url = new URLBuilder("api.example.com") {
    protocol "https"
    path "/users"
    param "active", true
}.build()
```

Chapter 21. Advanced Scenarios

21.1. Building URLs Programmatically

```
def buildSearchUrl(String host, Map filters) {
    def builder = new URLBuilder(host)
        .protocol("https")
        .path("/search")

    filters.each { key, value ->
        if (value != null) {
            builder.param(key, value)
        }
    }

    return builder.build()
}

def url = buildSearchUrl("api.example.com", [
    q: "groovy",
    limit: 10,
    tags: ["http", "client"]
])
```

21.2. Conditional Parameters

Only add parameters when needed:

```
def builder = new URLBuilder("api.example.com")
    .path("/articles")
    .param("limit", 10)

if (author) {
    builder.param("author", author)
}

if (tags) {
    builder.param("tags", tags)
}

if (since) {
    builder.param("since", since.format("yyyy-MM-dd"))
}

def url = builder.build()
```

21.3. URL Validation

Check if the built URL is valid:

```
def builder = new URLBuilder("api.example.com")
    .path("/users")

try {
    def url = builder.build()

    if (builder.isValidURL(url)) {
        // Proceed with URL
    }
} catch (IllegalStateException e) {
    // Invalid configuration
    println "URL build failed: ${e.message}"
}
```



`isValidURL()` performs basic URL syntax validation using `java.net.URI`.

Chapter 22. Integration with GroovyHttpClient

URLBuilder complements GroovyHttpClient perfectly:

```
import org.softwood.http.GroovyHttpClient
import org.softwood.url.URLBuilder

def baseUrl = new URLBuilder("api.github.com")
    .protocol("https")
    .build()

def client = new GroovyHttpClient(baseUrl)

// Build complex search URL
def searchPath = new URLBuilder()
    .path("/search/repositories")
    .param("q", "language:groovy")
    .param("sort", "stars")
    .param("order", "desc")
    .param("per_page", 10)
    .build()
    .replaceFirst("https?://[^/]+", "")  // Extract just the path+query

def response = client.getSync(searchPath)
def repos = response.json.items
```



For simple cases, use `GroovyHttpClient's built-in query DSL. For complex URL construction, use URLBuilder.

22.1. Using with Absolute URLs

```
def url = new URLBuilder("cdn.example.com")
    .protocol("https")
    .path("/files/document.pdf")
    .param("v", "1.2.3")
    .param("dl", 1)
    .build()

def config = SecurityConfig.production(baseApiUrl)
    .withOverrides([
        allowAbsoluteUrls: true,
        allowedHosts: ["api.example.com", "cdn.example.com"] as Set
    ])

def client = new GroovyHttpClient(config)
```

```
def bytes = client.downloadBytesSync(url)
```

Chapter 23. Character Encoding

URLBuilder uses UTF-8 encoding for all parameters:

```
def url = new URLBuilder("api.example.com")
    .path("/search")
    .param("q", "日本語")
    .param("emoji", "🚀")
    .build()

// Result:
https://api.example.com/search?q=%E6%97%A5%E6%9C%AC%E8%AA%9E&emoji=%F0%9F%9A%80
```

23.1. Special Characters

Common special characters are handled correctly:

Character	Example	Encoded
Space	`hello world`	`hello+world`
Plus	`a+b`	`a%2Bb`
Forward Slash	`path/to/file`	`path%2Fto%2Ffile`
Question Mark	`what?`	`what%3F`
Ampersand	`a&b`	`a%26b`
Equals	`x=y`	`x%3Dy`
Hash/Fragment	`#section`	`%23section`

Chapter 24. Error Handling

24.1. Missing Host

URLBuilder requires a host:

```
try {
    def url = new URLBuilder()  // No host!
        .path("/users")
        .build()
} catch (IllegalStateException e) {
    println e.message  // "Host is required to build a URL"
}
```

24.2. Invalid URI Syntax

Malformed URLs throw exceptions:

```
try {
    def url = new URLBuilder("not a valid host!!")
        .build()

    // Validate
    if (!new URLBuilder().isValidURL(url)) {
        throw new IllegalArgumentException("Invalid URL")
    }
} catch (URISyntaxException | MalformedURLException e) {
    println "URL validation failed: ${e.message}"
}
```

Chapter 25. Common Patterns

25.1. Pagination URLs

```
def buildPageUrl(String base, int page, int pageSize) {
    new URLBuilder(base)
        .protocol("https")
        .path("/articles")
        .param("page", page)
        .param("per_page", pageSize)
        .param("sort", "created_at")
        .param("order", "desc")
        .build()
}

def urls = (1..5).collect { page ->
    buildPageUrl("api.example.com", page, 20)
}
```

25.2. Filter URLs

```
class SearchUrlBuilder {
    private URLBuilder builder

    SearchUrlBuilder(String host) {
        this.builder = new URLBuilder(host)
            .protocol("https")
            .path("/search")
    }

    SearchUrlBuilder query(String q) {
        builder.param("q", q)
        this
    }

    SearchUrlBuilder tags(List<String> tags) {
        if (tags) {
            builder.param("tags", tags)
        }
        this
    }

    SearchUrlBuilder dateRange(String from, String to) {
        if (from) builder.param("from", from)
        if (to) builder.param("to", to)
        this
    }
```

```
    SearchUrlBuilder limit(int limit) {
        builder.param("limit", limit)
        this
    }

    String build() {
        builder.build()
    }
}

// Usage
def url = new SearchUrlBuilder("api.example.com")
    .query("groovy http")
    .tags(["http", "client"])
    .dateRange("2024-01-01", "2024-12-31")
    .limit(50)
    .build()
```

25.3. API Version URLs

```
def buildApiUrl(String host, String version, String endpoint) {
    new URLBuilder(host)
        .protocol("https")
        .path("/api/${version}${endpoint}")
        .build()
}

def url = buildApiUrl("api.example.com", "v2", "/users/123")
// → https://api.example.com/api/v2/users/123
```

Chapter 26. Testing with URLBuilder

URLBuilder simplifies test URL construction:

```
import spock.lang.Specification

class ApiClientSpec extends Specification {

    def "should build correct search URL"() {
        given:
        def builder = new URLBuilder("api.example.com")
            .path("/search")
            .param("q", "groovy")
            .param("limit", 10)

        when:
        def url = builder.build()

        then:
        url == "https://api.example.com/search?q=groovy&limit=10"
    }

    def "should handle array parameters"() {
        given:
        def tags = ["http", "client", "groovy"]

        when:
        def url = new URLBuilder("api.example.com")
            .path("/articles")
            .param("tags", tags)
            .build()

        then:
        url.contains("tags=http")
        url.contains("tags=client")
        url.contains("tags=groovy")
    }

    def "should encode special characters"() {
        given:
        def query = "hello world & friends"

        when:
        def url = new URLBuilder("api.example.com")
            .param("q", query)
            .build()

        then:
        url.contains("q=hello+world+%26+friends")
    }
```

```
}
```

Chapter 27. toString() and Build()

URLBuilder implements `toString()` to call `build()` automatically:

```
def builder = new URLBuilder("api.example.com")
    .path("/users")
    .param("active", true)

// These are equivalent:
def url1 = builder.build()
def url2 = builder.toString()
def url3 = "$builder"  // String interpolation

assert url1 == url2 && url2 == url3
```



This makes URLBuilder convenient in string contexts without explicit `build()` calls.

Chapter 28. Performance Considerations

URLBuilder is designed for construction, not repeated use:

Good — Build once, use many times:

```
def baseUrl = new URLBuilder("api.example.com")
    .protocol("https")
    .path("/api/v1")
    .build()

(1..1000).each { i ->
    def url = "${baseUrl}/users/${i}"
    // Make request with url
}
```

Avoid — Building in tight loops:

```
// ❌ Creates many builder instances
(1..1000).each { i ->
    def url = new URLBuilder("api.example.com")
        .path("/users/${i}")
        .build()
}
```



URLBuilder instances are lightweight, but excessive object creation in hot loops should be avoided.

Chapter 29. API Reference

29.1. Constructor Options

Constructor	Description
`URLBuilder()`	Default constructor — host must be set via `host()`
`URLBuilder(String host)`	Constructor with host
`URLBuilder(Closure config)`	Constructor with configuration closure

29.2. Fluent Methods

Method	Returns	Description
`protocol(String)`	`URLBuilder`	Set protocol (`http` or `https`)
`host(String)`	`URLBuilder`	Set hostname
`port(Integer)`	`URLBuilder`	Set port number
`path(String)`	`URLBuilder`	Set path segment
`param(String, Object)`	`URLBuilder`	Add single query parameter
`params(Map)`	`URLBuilder`	Add multiple query parameters
`build()`	`String`	Construct final URL string
`build(Closure)`	`String`	Apply config closure, then build
`toString()`	`String`	Same as `build()`
`isValidURL(String)`	`Boolean`	Validate URL syntax

29.3. Default Values

Property	Default
`protocol`	`https`
`port`	None (uses protocol default)
`path`	Empty string (root path)
`params`	Empty map

Chapter 30. Comparison with Alternatives

30.1. vs. String Concatenation

String Concat	URLBuilder
`def url = "https://" + host + ":" + port + "/search?q=" + URLEncoder.encode(query, "UTF-8")`	`def url = new URLBuilder(host).port(port).path("/search").param("q", query).build()`

URLBuilder Advantages:

- Automatic encoding
- Type safety
- Readable and maintainable
- Less error-prone

30.2. vs. URIBuilder (Apache HttpClient)

URLBuilder is simpler and Groovy-idiomatic:

URIBuilder	URLBuilder

`def builder = new URIBuilder().setScheme("https").setHost("api.example.com").setPath("/search").addParameter("q", "groovy")`
 	
`def url = builder.build().toString()`
	`def url = new URLBuilder ("api.example.com").path("/search").param("q", "groovy").build()`

URLBuilder Advantages:

- No external dependencies
- Groovy-friendly closure syntax
- Simpler API
- Natural array parameter handling

Chapter 31. Best Practices

31.1. Validate Required Parameters

```
def buildUserUrl(String userId) {
    Objects.requireNonNull(userId, "userId is required")

    new URLBuilder("api.example.com")
        .path("/users/${userId}")
        .build()
}
```

31.2. Use Factory Methods

```
class UrlFactory {
    static String userUrl(String userId) {
        new URLBuilder("api.example.com")
            .path("/users/${userId}")
            .build()
    }

    static String searchUrl(String query, int limit = 10) {
        new URLBuilder("api.example.com")
            .path("/search")
            .param("q", query)
            .param("limit", limit)
            .build()
    }
}

// Usage
def url = UrlFactory.searchUrl("groovy http client")
```

31.3. Document URL Structure

```
/**
 * Builds a paginated article list URL.
 *
 * @param page Page number (1-based)
 * @param pageSize Items per page
 * @param tags Optional tag filters
 * @return Full URL with query parameters
 */
String buildArticleListUrl(int page, int pageSize, List<String> tags = []) {
    def builder = new URLBuilder("api.example.com")
```

```
        .path("/articles")
        .param("page", page)
        .param("per_page", pageSize)

    if (tags) {
        builder.param("tags", tags)
    }

    builder.build()
}
```

Chapter 32. Next Steps

Now that you understand URL construction, learn about:

- GroovyHttpClient Deep Dive — Making requests with built URLs
- Multipart Uploads — Sending files and form data
- Testing with MockHttpServer — Mocking URL-based endpoints

GroovyHttpClient Deep Dive

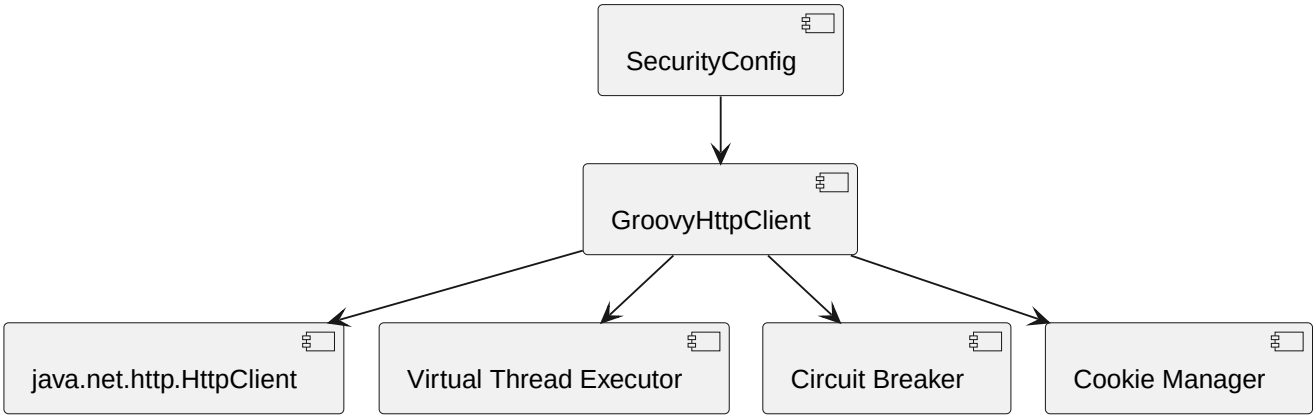
Chapter 33. Overview

`GroovyHttpClient` is the core class of this library, providing a fluent Groovy DSL for making HTTP requests. Built on Java 21's `HttpClient` and virtual threads, it delivers enterprise-grade features with an elegant, concise API.

> HTTP requests should read like natural language, not cryptic API calls.

33.1. Architecture

At its core, GroovyHttpClient wraps three key Java components:





Unlike libraries that replace Java's HTTP stack, GroovyHttpClient **wraps** it, ensuring compatibility and leveraging battle-tested HTTP/2 implementation.

Chapter 34. Creating a Client

34.1. Simple Construction

The simplest way to create a client:

```
import org.softwood.http.GroovyHttpClient

def client = new GroovyHttpClient("https://api.example.com")
```

This uses testing defaults (relaxed security, suitable for development).

34.2. With SecurityConfig

For production use, always provide explicit configuration:

```
import org.softwood.http.SecurityConfig
import org.softwood.http.GroovyHttpClient

def config = SecurityConfig.production("https://api.example.com")
def client = new GroovyHttpClient(config)
```

34.3. Using Builder

For fine-grained control:

```
def client = GroovyHttpClient.builder("https://api.example.com")
    .connectTimeout(Duration.ofSeconds(5))
    .requestTimeout(Duration.ofSeconds(30))
    .allowRedirects(true)
    .failureThreshold(5)
    .maxResponseBytes(50_000_000L)
    .build()
```



The builder approach is convenient when you need to customize just a few settings without creating a full `SecurityConfig`.

Chapter 35. HTTP Methods

GroovyHttpClient supports all standard HTTP methods with both synchronous and asynchronous APIs.

35.1. GET Requests

Synchronous:

```
def response = client.getSync("/users/123")

println "Status: ${response.statusCode}"
println "Body: ${response.body}"
```

Asynchronous:

```
import java.util.concurrent.CompletableFuture

CompletableFuture<HttpClientResponse> future = client.get("/users/123")

future.thenAccept { response ->
    println "Got response: ${response.statusCode}"
}.exceptionally { error ->
    println "Request failed: ${error.message}"
    null
}
```

35.2. POST Requests

With String Body:

```
def jsonBody = '{"name": "Alice", "email": "alice@example.com"}'

def response = client.postSync("/users", jsonBody) {
    header "Content-Type", "application/json"
}
```

With JSON DSL:

```
def response = client.postSync("/users", null) {
    json([
        name: "Alice",
        email: "alice@example.com",
        roles: ["user", "admin"]
    ])
```

```
}
```



When using the `json { }` DSL, the `Content-Type: application/json` header is set automatically.

35.3. PUT Requests

```
def response = client.putSync("/users/123", null) {
    json([
        name: "Alice Smith",
        email: "alice.smith@example.com"
    ])
}

assert response.statusCode == 200
```

35.4. PATCH Requests

```
def response = client.patchSync("/users/123", null) {
    json([email: "newemail@example.com"])
}
```

35.5. DELETE Requests

```
def response = client.deleteSync("/users/123")
assert response.statusCode == 204
```

35.6. HEAD Requests

```
def response = client.headSync("/users/123")

// Check if resource exists
if (response.statusCode == 200) {
    println "Content-Length: ${response.getHeader('Content-Length')}"
}
```

35.7. OPTIONS Requests

```
def response = client.optionsSync("/users")
```

```
def allowedMethods = response.getHeader("Allow")
println "Supported methods: $allowedMethods"
```

Chapter 36. Request Configuration DSL

All request methods accept an optional configuration closure:

```
def response = client.getSync("/search") {
    // Query parameters
    query(q: "groovy", limit: 10)

    // Headers
    header "Accept", "application/json"
    header "X-API-Key", apiKey

    // Timeout override
    timeout Duration.ofSeconds(60)

    // Cookies
    cookie "session", sessionId

    // Retry override
    retry {
        maxAttempts 5
        backoffExponential(500, 5000, true)
    }
}
```

36.1. Query Parameters

Single Parameters:

```
client.getSync("/search") {
    query(q: "groovy", limit: 10, offset: 0)
}
// → /search?q=groovy&limit=10&offset=0
```

Array Parameters:

```
client.getSync("/articles") {
    query(tags: ["groovy", "http", "client"])
}
// → /articles?tags=groovy&tags=http&tags=client
```

Map of Lists:

```
def params = [
    status: ["published", "draft"],
```

```
    author: ["alice"]
]

client.getSync("/articles") {
    query(params)
}
// → /articles?status=published&status=draft&author=alice
```



Query parameters are automatically URL-encoded. Don't encode them yourself or they'll be double-encoded.

36.2. Headers

Single Header:

```
client.getSync("/data") {
    header "Accept", "application/json"
}
```

Multiple Values:

```
client.getSync("/data") {
    header "Accept", "application/json"
    header "Accept-Encoding", "gzip, deflate"
    header "X-Custom", "value1"
    header "X-Custom", "value2"  // Multiple values for same header
}
```

Conditional Headers:

```
client.getSync("/data") {
    if (useAuth) {
        header "Authorization", "Bearer $token"
    }
    if (acceptCompression) {
        header "Accept-Encoding", "gzip"
    }
}
```

36.3. Cookies

Single Cookie:

```
client.getSync("/profile") {
```

```
    cookie "session", "abc123"
}
```

Multiple Cookies:

```
client.getSync("/profile") {
    cookies(
        session: "abc123",
        preferences: "dark-mode",
        language: "en"
    )
}
```



Cookies set via the DSL are **per-request**. For persistent cookies across requests, use the automatic cookie manager (see Cookie Management).

36.4. Form Data

URL-Encoded Forms:

```
def response = client.postSync("/login", null) {
    form(
        username: "alice",
        password: "secret123",
        remember: "true"
    )
}
// Content-Type: application/x-www-form-urlencoded
```

With Custom Charset:

```
import java.nio.charset.Charset

client.postSync("/data", null) {
    form(
        [text: "□□□□□"],
        Charset.forName("UTF-8")
    )
}
```

36.5. JSON Bodies

From Map:

```
client.postSync("/users", null) {
    json([
        name: "Alice",
        age: 30,
        roles: ["admin", "user"],
        metadata: [
            created: "2025-01-24",
            source: "api"
        ]
    ])
}
```

From String:

```
def jsonString = '{"name":"Alice","age":30}'

client.postSync("/users", null) {
    json(jsonString)
}
```

From Object:

```
class User {
    String name
    int age
    List<String> roles
}

def user = new User(name: "Alice", age: 30, roles: ["admin"])

client.postSync("/users", null) {
    json(user)  // Uses JsonOutput.toJson()
}
```

36.6. Custom Request Timeout

Override the default request timeout:

```
// Default: 30 seconds (from SecurityConfig)
def response = client.getSync("/fast-endpoint")

// Override: 5 seconds for this request
def response2 = client.getSync("/fast-endpoint") {
    timeout Duration.ofSeconds(5)
}
```

```
// Override: 2 minutes for slow endpoint
def response3 = client.getSync("/slow-report") {
    timeout Duration.ofMinutes(2)
}
```



Setting very long timeouts can cause resource exhaustion under load. Use judiciously and consider circuit breaker implications.

Chapter 37. Response Handling

37.1. Response Object

All requests return an `HttpClientResponse` object:

```
def response = client.getSync("/users/123")

// Properties
int statusCode = response.statusCode
String body = response.body
byte[] bytes = response.bytes
Map<String, List<String>> headers = response.headers
String contentType = response.contentType
Charset charset = response.charset
```

37.2. Status Code Checks

```
if (response.isSuccess()) {
    println "Success! (2xx)"
} else if (response.isClientError()) {
    println "Client error (4xx)"
} else if (response.isServerError()) {
    println "Server error (5xx)"
}

// Direct checks
if (response.statusCode == 404) {
    println "Not found"
}
```

Helper methods: * `isSuccess()` — 200-299 * `isClientError()` — 400-499 * `isServerError()` — 500-599

37.3. Header Access

```
// Get single header value
String contentType = response.getHeader("Content-Type")

// Get all values for header (multi-value headers)
List<String> setCookies = response.getHeaders("Set-Cookie")

// Check if header exists
if (response.hasHeader("ETag")) {
    String etag = response.getHeader("ETag")
}
```

```
// Case-insensitive lookup
String auth = response.getHeader("AUTHORIZATION")  // Works
```

37.4. JSON Responses

Automatic Parsing:

```
def response = client.getSync("/users/123")

// Requires Content-Type: application/json
def user = response.json

println "Name: ${user.name}"
println "Email: ${user.email}"
```

Safe Parsing:

```
def json = response.jsonOrNull

if (json) {
    // Successfully parsed
} else {
    // Not JSON or parse failed
    println "Response was not JSON"
}
```

Require JSON:

```
try {
    def user = response.requireJson()
    // Throws if not JSON
} catch (ContentParseException e) {
    println "Expected JSON but got: ${response.contentType}"
}
```



JSON parsing is **content-type aware**. The response must have:

- `Content-Type: application/json`
- `Content-Type: application/vnd.api+json`
- Or any type containing `+json`

Otherwise, `json` property throws `ContentParseException`.

37.5. XML Responses

Similar to JSON, but for XML:

```
def response = client.getSync("/users/123.xml")

def user = response.xml
println "Name: ${user.name.text()}"

// Or safe parsing
def xmlOrNull = response.xmlOrNull
if (xmlOrNull) {
    // Parse successful
}

// Or require
def xmlRequired = response.requireXml()
```

XML is parsed using Groovy's `XmlSlurper` and requires:

- `Content-Type: application/xml`
- `Content-Type: text/xml`
- Or any type containing `+xml`

37.6. Binary Responses

Access raw bytes:

```
def response = client.getSync("/files/document.pdf")

byte[] pdfBytes = response.bytes

// Save to file
new File("document.pdf").bytes = pdfBytes

// Get size
println "Downloaded ${pdfBytes.length} bytes"
```

Chapter 38. Cookie Management

GroovyHttpClient includes automatic cookie management via `java.net.CookieManager`.

38.1. Automatic Cookies

Cookies from responses are automatically stored and sent on subsequent requests:

```
// Server sends: Set-Cookie: session=abc123; Path=/
def login = client.postSync("/login", null) {
    form(username: "alice", password: "secret")
}

// Cookie automatically included in next request
def profile = client.getSync("/profile")
// → Cookie: session=abc123
```

38.2. Adding Cookies

Fluent API:

```
client.addCookie("session", "abc123", "/")
client.addCookie("preferences", "dark-mode", "/")

def response = client.getSync("/profile")
// Cookies sent automatically
```

Using HttpCookie:

```
import java.net.HttpCookie

def cookie = new HttpCookie("session", "abc123")
cookie.path = "/"
cookie.domain = "api.example.com"
cookie.maxAge = 3600
cookie.secure = true

client.addCookie(cookie)
```

38.3. Retrieving Cookies

```
// Get all cookies for this client's host
List<HttpCookie> cookies = client.cookies
```

```
cookies.each { cookie ->
    println "${cookie.name} = ${cookie.value}"
}

// Get specific cookie
HttpCookie session = client.getCookie("session")
if (session) {
    println "Session: ${session.value}"
}
```

38.4. Removing Cookies

```
// Remove specific cookie
client.removeCookie("session")

// Clear all cookies
client.clearCookies()
```

38.5. Cookie Policies

Change how cookies are accepted:

```
import java.net.CookiePolicy

// Accept all cookies
client.setCookiePolicy(CookiePolicy.ACCEPT_ALL)

// Accept only from original server (default)
client.setCookiePolicy(CookiePolicy.ACCEPT_ORIGINAL_SERVER)

// Reject all cookies
client.setCookiePolicy(CookiePolicy.ACCEPT_NONE)

// Get current policy
CookiePolicy policy = client.cookiePolicy
```



For testing scenarios with multiple domains, use `ACCEPT_ALL`. For production, use `ACCEPT_ORIGINAL_SERVER` (default).

Chapter 39. Default Headers

Set headers that apply to all requests:

39.1. Setting Default Headers

Single Header:

```
client.setHeader("User-Agent", "MyApp/1.0")
client.setHeader("Accept", "application/json")

// Now all requests include these headers
def response = client.getSync("/data")
```

Multiple Headers:

```
client.setHeaders([
    "User-Agent": "MyApp/1.0",
    "Accept": "application/json",
    "Accept-Encoding": "gzip"
])
```

39.2. Adding to Default Headers

```
// Set initial header
client.setHeader("X-App-Version", "1.0")

// Add another value for same header
client.withHeader("X-App-Version", "1.0.1")

// Result: X-App-Version sent with both values
```

39.3. Clearing Default Headers

```
client.clearHeaders()
```

39.4. Viewing Default Headers

```
Map<String, List<String>> headers = client.defaultHeaders

headers.each { name, values ->
    println "$name: ${values.join(', ')}"
```

```
}
```



Per-request headers (set in DSL) **override** default headers with the same name. They don't merge.

```
client.setHeader("Accept", "application/json")

// This request uses text/html, not application/json
client.getSync("/page") {
    header "Accept", "text/html"
}
```

Chapter 40. Downloading Files

40.1. Binary Downloads

Synchronous:

```
byte[] fileData = client.downloadBytesSync("/files/document.pdf")

new File("document.pdf").bytes = fileData
```

Asynchronous:

```
client.downloadBytes("/files/large-file.zip").thenAccept { bytes ->
    new File("large-file.zip").bytes = bytes
    println "Downloaded ${bytes.length} bytes"
}
```

40.2. With Custom Timeout

```
// 5 minute timeout for large download
byte[] data = client.downloadBytesSync(
    "/files/huge-file.zip",
    Duration.ofMinutes(5)
)
```

40.3. Streaming Large Files

For very large files, avoid loading entirely into memory:

```
def response = client.getSync("/files/huge-file.zip")

new File("huge-file.zip").withOutputStream { out ->
    out.write(response.bytes)
}
```



`downloadBytes()` and `response.bytes` load the entire response into memory. For files exceeding a few hundred MB, consider using Java's `HttpClient` directly with `BodyHandlers.ofInputStream()`.

Chapter 41. Error Handling

41.1. HTTP Error Responses

By default, responses with status >= 400 throw `HttpResponseException`:

```
try {
    def response = client.getSync("/nonexistent")
} catch (GroovyHttpClient.HttpResponseException e) {
    println "HTTP Error ${e.statusCode}"
    println "Response body: ${e.message}"

    if (e.statusCode == 404) {
        println "Resource not found"
    } else if (e.statusCode >= 500) {
        println "Server error"
    }
}
```

41.2. Network Errors

```
import java.io.IOException
import java.net.http.HttpTimeoutException

try {
    def response = client.getSync("/data")
} catch (HttpTimeoutException e) {
    println "Request timed out"
} catch (IOException e) {
    println "Network error: ${e.message}"
} catch (InterruptedException e) {
    println "Request interrupted"
}
```

41.3. Circuit Breaker Errors

```
try {
    def response = client.getSync("/data")
} catch (GroovyHttpClient.CircuitOpenException e) {
    println "Circuit breaker is open - service unavailable"
    // Handle gracefully: return cached data, fallback, etc.
}
```

41.4. Content Parse Errors

```
try {
    def data = response.json
} catch (GroovyHttpClient.ContentParseException e) {
    println "Failed to parse JSON: ${e.message}"
    println "Content-Type was: ${response.contentType}"
}
```

Chapter 42. Retry Policies

Configure automatic retries for failed requests.

42.1. Per-Client Retry Policy

Set via `SecurityConfig`:

```
import org.softwood.http.GroovyHttpClient.RetryPolicy

def retryPolicy = new RetryPolicy.Config().tap {
    maxAttempts(3)
    backoffExponential(200, 2000, true)
    onIoExceptions(true)
    onTimeouts(true)
    on5xx(true)
}.build()

def config = SecurityConfig.production("https://api.example.com")
    .withOverrides([retryPolicy: retryPolicy])

def client = new GroovyHttpClient(config)
```

42.2. Per-Request Retry Policy

Override retry behavior for specific requests:

```
def response = client.getSync("/critical-data") {
    retry {
        maxAttempts 5
        backoffExponential(500, 10000, true)
        onIoExceptions(true)
        onTimeouts(true)
        on5xx(true)
    }
}
```

42.3. Retry Conditions

On I/O Exceptions:

```
retry {
    onIoExceptions(true)  // Connection failures, etc.
}
```

On Timeouts:

```
retry {
    onTimeouts(true)  // Request timeout exceeded
}
```

On 5xx Errors:

```
retry {
    on5xx(true)  // All 500-599 status codes
}
```

On Specific Status Codes:

```
retry {
    onStatusCodes([502, 503, 504])  // Only these codes
}
```

42.4. Backoff Strategies

Exponential Backoff with Jitter:

```
retry {
    maxAttempts(4)
    // Start at 100ms, double each time, max 8s, with jitter
    backoffExponential(100, 8000, true)
}
// Delays: ~50-100ms, ~100-200ms, ~200-400ms, ~400-800ms
```

Fixed Delay:

```
retry {
    maxAttempts(3)
    // Always wait 1 second
    backoffExponential(1000, 1000, false)
}
```

No Retries:

```
retry {
    maxAttempts(1)  // Original attempt only
}
```



Be very careful with retries on POST, PUT, PATCH, DELETE. These operations are typically **not idempotent**, so retrying can cause:

- Duplicate charges
- Multiple record creations
- Data inconsistency

For non-idempotent operations, implement idempotency tokens or use `maxAttempts(1)`.

Chapter 43. Request Interceptors

Interceptors provide middleware-style request/response processing.

43.1. Adding Interceptors

```
client.addInterceptor { ctx, next ->
    println "→ ${ctx.method} ${ctx.uri}"

    return next.call(ctx).thenApply { response ->
        println "← ${response.statusCode}"
        response
    }
}
```

43.2. Common Use Cases

Logging Interceptor:

```
client.addInterceptor { ctx, next ->
    def start = System.currentTimeMillis()

    return next.call(ctx).thenApply { response ->
        def elapsed = System.currentTimeMillis() - start
        println "${ctx.method} ${ctx.uri} → ${response.statusCode} (${elapsed}ms)"
        response
    }
}
```

Authentication Interceptor:

```
def authToken = "Bearer xyz123"

client.addInterceptor { ctx, next ->
    // Add auth header to every request
    def request = ctx.requestSupplier.call()
    def newRequest = HttpRequest.newBuilder(request, { builder, name, value ->
        builder.header(name, value)
    })
    .header("Authorization", authToken)
    .build()

    // Update context
    ctx.requestSupplier = { newRequest }

    return next.call(ctx)
```

```
}
```

Retry Interceptor (custom logic):

```
client.addInterceptor { ctx, next ->
    int attempt = 0
    CompletableFuture<HttpClientResponse> result

    while (attempt < 3) {
        result = next.call(ctx)
        try {
            def response = result.get()
            if (response.statusCode < 500) {
                return CompletableFuture.completedFuture(response)
            }
        } catch (Exception e) {
            // Retry on exception
        }
        attempt++
        Thread.sleep(100 * attempt)
    }

    return result
}
```

Metrics Interceptor:

```
import java.util.concurrent.atomic.AtomicInteger

def requestCount = new AtomicInteger(0)
def errorCount = new AtomicInteger(0)

client.addInterceptor { ctx, next ->
    requestCount.incrementAndGet()

    return next.call(ctx).whenComplete { response, error ->
        if (error) {
            errorCount.incrementAndGet()
        }
    }
}

// Check metrics
println "Requests: ${requestCount.get()}, Errors: ${errorCount.get()}"
```

43.3. Interceptor Context

The `RequestContext` object provides:

Property	Description
`method`	HTTP method (GET, POST, etc.)
`uri`	Target URI
`requestSupplier`	Closure that builds HttpRequest
`retryPolicy`	Active retry policy for this request
`attempt`	Current attempt number (1-based)
`proceed()`	Optional method to call next interceptor



Interceptors execute in LIFO order (last added runs first). This matches how middleware chains typically work.

Chapter 44. Lifecycle Management

GroovyHttpClient implements `AutoCloseable` for resource management.

44.1. Manual Cleanup

```
def client = new GroovyHttpClient("https://api.example.com")

try {
    def response = client.getSync("/data")
    // ... use response
} finally {
    client.close()  // Cleanup executor
}
```

44.2. Try-With-Resources

```
new GroovyHttpClient("https://api.example.com").withCloseable { client ->
    def response = client.getSync("/data")
    println response.body
}
// Automatically closed
```

44.3. Long-Lived Clients

For application-scoped clients:

```
@Singleton
class ApiClient {
    private final GroovyHttpClient client

    ApiClient() {
        def config = SecurityConfig.production("https://api.example.com")
        this.client = new GroovyHttpClient(config)
    }

    def getUser(String id) {
        client.getSync("/users/${id}")
    }

    void shutdown() {
        client.close()
    }
}
```

```
// Register shutdown hook
Runtime.runtime.addShutdownHook {
    ApiClient.instance.shutdown()
}
```



Don’t create a new client for every request. Client creation is relatively expensive (thread pool setup, SSL context initialization). Reuse clients across requests.

Chapter 45. Next Steps

Continue learning about advanced features:

- Multipart Uploads — Sending files and forms
- Circuit Breaker Details — Understanding resilience
- MockHttpServer — Testing HTTP interactions
- HttpClientTestDsl — Fluent test DSL
- Real-World Examples — Practical usage patterns

Multipart Uploads

Chapter 46. Overview

GroovyHttpClient provides comprehensive support for `multipart/form-data` uploads, enabling you to send files, binary data, and form fields in a single HTTP request. This is essential for file uploads, mixed content submission, and API integrations requiring complex payloads.

> Multipart uploads should be as simple as regular POST requests — no manual boundary management or byte manipulation required.

46.1. Key Features

- □ Two API styles: DSL-based and static builder
- □ Automatic boundary generation
- □ Support for text, JSON, and binary parts
- □ Custom content types per part
- □ Filename and encoding control
- □ Request header customization
- □ Asynchronous and synchronous operations
- □ Integration with retry and circuit breaker

Chapter 47. Understanding Multipart Requests

A multipart request consists of multiple "parts," each with:

Name	Form field name (required)
Filename	Optional filename for file uploads
Content-Type	MIME type of the part data
Data	Binary content (bytes)

The HTTP client automatically handles:

- Boundary generation and formatting
- Content-Disposition headers
- CRLF line endings
- Proper Content-Type with boundary parameter

Chapter 48. DSL-Based Multipart

The DSL provides an intuitive, builder-style API for constructing multipart requests.

48.1. Basic DSL Upload

```
def response = client.postMultipartSync("/upload") {
    part {
        name "document"
        filename "report.pdf"
        contentType "application/pdf"
        data pdfBytes
    }

    part {
        name "description"
        content "Monthly sales report"
    }
}

assert response.statusCode == 201
```

48.2. Text Parts

Use the `content` method for text data:

```
client.postMultipartSync("/submit") {
    part {
        name "username"
        content "alice"
    }

    part {
        name "comment"
        content "This is a multiline\ncomment with special chars: é ñ ü"
    }
}
```



The `content` method automatically:

- Converts text to UTF-8 bytes
- Sets `Content-Type: text/plain; charset=UTF-8`
- Generates a filename if not specified

48.3. Binary Parts

For binary data, use the `data` method:

```
def imageBytes = new File("photo.jpg").bytes

client.postMultipartSync("/photos") {
    part {
        name "photo"
        filename "vacation.jpg"
        contentType "image/jpeg"
        data imageBytes
    }

    part {
        name "title"
        content "Summer Vacation 2025"
    }
}
```

48.4. JSON Parts

Submit JSON as part of a multipart request:

```
def metadata = [
    author: "Alice",
    tags: ["important", "reviewed"],
    date: "2025-01-24"
]

client.postMultipartSync("/documents") {
    part {
        name "file"
        filename "data.csv"
        contentType "text/csv"
        data csvBytes
    }

    part {
        name "metadata"
        filename "metadata.json"
        contentType "application/json; charset=UTF-8"
        content groovy.json.JsonOutput.toJson(metadata)
    }
}
```

48.5. Multiple Files

Upload multiple files in a single request:

```
def files = [
    [name: "file1.txt", bytes: file1Bytes],
    [name: "file2.pdf", bytes: file2Bytes],
    [name: "file3.jpg", bytes: file3Bytes]
]

client.postMultipartSync("/batch-upload") {
    files.each { f ->
        part {
            name "files"  // Same name for all parts
            filename f.name
            contentType guessMimeType(f.name)
            data f.bytes
        }
    }

    part {
        name "count"
        content files.size().toString()
    }
}
```



Multiple parts with the same `name` are valid and represent array/multiple values — common for file upload APIs.

48.6. Adding Request Headers

Include custom headers alongside multipart data:

```
client.postMultipartSync("/secure-upload") {
    part {
        name "file"
        filename "secret.zip"
        contentType "application/zip"
        data zipBytes
    }

    // Request headers (not part headers)
    header "Authorization", "Bearer ${authToken}"
    header "X-Request-ID", UUID.randomUUID().toString()
}
```

48.7. Asynchronous DSL Upload

For non-blocking uploads:

```
import java.util.concurrent.CompletableFuture

CompletableFuture<HttpClientResponse> future = client.postMultipart("/upload") {
    part {
        name "document"
        filename "report.pdf"
        data pdfBytes
    }
}

future.thenAccept { response ->
    println "Upload complete: ${response.statusCode}"
    println "Response: ${response.json}"
}.exceptionally { error ->
    println "Upload failed: ${error.message}"
    null
}
```

Chapter 49. Static Builder API

For programmatic construction, use `MultipartPart` static methods.

49.1. Text Parts

```
import org.softwood.http.MultipartPart

def parts = [
    MultipartPart.text("username", "alice"),
    MultipartPart.text("email", "alice@example.com")
]

def response = client.postMultipartSync("/register", parts)
```

49.2. JSON Parts

```
def userData = [
    name: "Alice Smith",
    age: 30,
    roles: ["admin", "user"]
]

def parts = [
    MultipartPart.json("user", groovy.json.JsonOutput.toJson(userData))
]

client.postMultipartSync("/users", parts)
```

49.3. Binary Parts

```
def imageData = new File("photo.jpg").bytes

def parts = [
    MultipartPart.bytes("photo", "vacation.jpg", "image/jpeg", imageData),
    MultipartPart.text("caption", "Beautiful sunset")
]

client.postMultipartSync("/photos", parts)
```

49.4. Mixed Content

Combine different part types:

```
def csvData = new File("data.csv").bytes
def metadata = [processed: true, rows: 1000]

def parts = [
    MultipartPart.bytes("file", "data.csv", "text/csv", csvData),
    MultipartPart.json("metadata", groovy.json.JsonOutput.toJson(metadata)),
    MultipartPart.text("user_id", "12345")
]

def response = client.postMultipartSync("/import", parts)
assert response.statusCode == 200
```

49.5. With Request Configuration

Add headers or other configuration with the static API:

```
def parts = [
    MultipartPart.bytes("file", "secret.zip", "application/zip", zipBytes)
]

def response = client.postMultipartSync("/secure-upload", parts) { config ->
    config.header "Authorization", "Bearer ${token}"
    config.header "X-Correlation-ID", correlationId

    config.timeout Duration.ofMinutes(5)  // Long upload timeout
}
```

Chapter 50. Advanced Patterns

50.1. Large File Streaming

For very large files, load data incrementally:

```
// Read file in chunks
def file = new File("large-video.mp4")
def fileBytes = file.bytes  // Load entire file (consider streaming for huge files)

client.postMultipartSync("/videos", [
    MultipartPart.bytes("video", file.name, "video/mp4", fileBytes)
]) { config ->
    config.timeout Duration.ofMinutes(30)  // Extended timeout
}
```



The current implementation loads all part data into memory. For files exceeding available RAM, consider:

- Splitting into smaller chunks
- Using direct Java `HttpClient` with streaming
- Implementing server-side chunked upload support

50.2. Progress Tracking

While GroovyHttpClient doesn't have built-in progress tracking, you can implement it via interceptors:

```
import java.util.concurrent.atomic.AtomicLong

def uploadedBytes = new AtomicLong(0)
def totalBytes = calculateTotalSize(parts)

client.addInterceptor { ctx, next ->
    // Track upload progress
    def startTime = System.currentTimeMillis()

    next.call(ctx).thenApply { response ->
        def elapsed = System.currentTimeMillis() - startTime
        println "Upload complete: ${totalBytes} bytes in ${elapsed}ms"
        response
    }
}

client.postMultipartSync("/upload", parts)
```

50.3. Dynamic Part Generation

Generate parts dynamically based on runtime conditions:

```
def files = findFilesToUpload()

client.postMultipartSync("/batch") {
    // Add metadata part
    part {
        name "metadata"
        contentType "application/json"
        content groovy.json.JsonOutput.toJson([
            count: files.size(),
            timestamp: System.currentTimeMillis()
        ])
    }

    // Add file parts dynamically
    files.eachWithIndex { file, idx ->
        part {
            name "file_${idx}"
            filename file.name
            contentType guessMimeType(file.name)
            data file.bytes
        }
    }
}
```

50.4. Conditional Part Inclusion

Include parts based on conditions:

```
client.postMultipartSync("/submit") {
    part {
        name "required_field"
        content "always present"
    }

    if (includeOptionalFile) {
        part {
            name "optional_file"
            filename "extra.pdf"
            data optionalBytes
        }
    }

    if (metadata) {
        part {
```

```
            name "metadata"
            contentType "application/json"
            content groovy.json.JsonOutput.toJson(metadata)
        }
    }
}
```

Chapter 51. Content Type Reference

Common MIME types for multipart uploads:

File Type	Content-Type
Plain Text	`text/plain; charset=UTF-8`
JSON	`application/json; charset=UTF-8`
CSV	`text/csv; charset=UTF-8`
PDF	`application/pdf`
ZIP Archive	`application/zip`
JPEG Image	`image/jpeg`
PNG Image	`image/png`
MP4 Video	`video/mp4`
Binary Data	`application/octet-stream`
XML	`application/xml; charset=UTF-8`

Chapter 52. Error Handling

52.1. Upload Failures

```
try {
    def response = client.postMultipartSync("/upload") {
        part {
            name "file"
            filename "data.csv"
            data csvBytes
        }
    }

    println "Upload successful: ${response.json}"

} catch (GroovyHttpClient.HttpResponseException e) {
    if (e.statusCode == 413) {
        println "File too large"
    } else if (e.statusCode == 400) {
        println "Invalid file format"
    } else {
        println "Upload failed: ${e.message}"
    }
} catch (IOException e) {
    println "Network error: ${e.message}"
}
```

52.2. Validation Errors

Handle server-side validation:

```
def response = client.postMultipartSync("/upload") {
    part {
        name "file"
        filename "data.txt"
        data fileBytes
    }
}

if (!response.isSuccess()) {
    def errors = response.json.errors
    println "Validation failed:"
    errors.each { field, message ->
        println "  ${field}: ${message}"
    }
}
```

52.3. Retry Configuration

Configure retries for flaky uploads:

```
client.postMultipartSync("/upload") {
    part {
        name "file"
        filename "important.pdf"
        data pdfBytes
    }

    retry {
        maxAttempts 3
        backoffExponential(1000, 5000, true)
        onTimeouts true
        on5xx true
    }
}
```

Chapter 53. Testing Multipart Uploads

53.1. With HttpClientTestDsl

```
def testDsl = HttpClientTestDsl.create()
testDsl.start()

testDsl.when("/upload")
    .respondWith {
        status 201
        json([uploaded: true, filename: "test.txt"])
    }

def client = new GroovyHttpClient(testDsl.baseUrl)

def response = client.postMultipartSync("/upload") {
    part {
        name "file"
        filename "test.txt"
        content "Hello, World!"
    }
}

assert response.statusCode == 201
assert response.json.uploaded == true

testDsl.verify("/upload") {
    wasCalled(1)
    withMethod("POST")
    withHeader("Content-Type")  // Contains multipart/form-data
}

testDsl.stop()
```

53.2. With MockHttpServer

```
def server = new MockHttpServer()
server.init()

server.addRequestCheck("POST", "/upload", 200)
    .withExpectedMultipart([
        [name: "file", filename: "test.txt", content: "Hello!".bytes],
        [name: "description", content: "Test file".bytes]
    ])
    .withResponseBody('{"success":true}')

def client = new GroovyHttpClient("http://localhost:${server.port}")
```

```
def response = client.postMultipartSync("/upload") {
    part {
        name "file"
        filename "test.txt"
        content "Hello!"
    }
    part {
        name "description"
        content "Test file"
    }
}

assert response.statusCode == 200

server.shutdown()
```



MockHttpServer validates multipart requests by checking:

- Part names match expected
- Filenames match (if specified in expectations)
- Content matches byte-for-byte
- Content-Types match (if specified)

Chapter 54. Best Practices

54.1. Choose the Right API

DSL	Inline, ad-hoc uploads with few parts
Static	Programmatic, reusable upload logic
Hybrid	Static parts + DSL configuration

```
// ✅ DSL: Simple, inline
client.postMultipartSync("/upload") {
    part { name "file"; filename "doc.pdf"; data bytes }
}

// ✅ Static: Reusable, composable
def parts = buildUploadParts(files)
client.postMultipartSync("/upload", parts)

// ✅ Hybrid: Combine both
client.postMultipartSync("/upload", parts) { config ->
    config.header "Authorization", token
}
```

54.2. Always Set Content-Type

```
// ✅ Explicit content type
part {
    name "file"
    filename "data.csv"
    contentType "text/csv; charset=UTF-8"
    data csvBytes
}

// ❌ Missing content type (defaults to text/plain)
part {
    name "file"
    data csvBytes
}
```

54.3. Use Meaningful Filenames

```
// ✅ Descriptive filename
part {
    name "invoice"
```

```
    filename "invoice_2025_01_24.pdf"
    data pdfBytes
}

// ✅ Generic filename
part {
    name "invoice"
    filename "file.pdf"  // Not helpful for debugging
    data pdfBytes
}
```

54.4. Handle Large Files Appropriately

```
// ✅ Extended timeout for large files
client.postMultipartSync("/videos", parts) { config ->
    config.timeout Duration.ofMinutes(10)
}

// ❌ Default timeout may be too short
client.postMultipartSync("/videos", parts)
```

54.5. Validate Before Upload

```
// ✅ Validate locally
def file = new File("upload.pdf")
if (!file.exists()) {
    throw new IllegalArgumentException("File not found")
}
if (file.length() > 100_000_000) {
    throw new IllegalArgumentException("File too large (>100MB)")
}
if (!file.name.endsWith(".pdf")) {
    throw new IllegalArgumentException("Only PDF files allowed")
}

// Now upload
def response = client.postMultipartSync("/documents") {
    part {
        name "file"
        filename file.name
        contentType "application/pdf"
        data file.bytes
    }
}
```

Chapter 55. Troubleshooting

55.1. "Multipart parts are required" Error

```
// ❌ Fails: Empty parts list
client.postMultipartSync("/upload", [])

// ✅ Fixed: At least one part
client.postMultipartSync("/upload", [
    MultipartPart.text("field", "value")
])
```

55.2. Missing Filename

```
// The DSL auto-generates a filename if omitted
part {
    name "field"
    content "text data"
    // filename automatically becomes "field.txt"
}

// For the static API, always provide filename for binary parts
MultipartPart.bytes("file", "document.pdf", "application/pdf", bytes)
```

55.3. Content-Type Issues

```
// ✅ Correct: Include charset for text
part {
    name "description"
    contentType "text/plain; charset=UTF-8"
    content "Description text"
}

// ❌ Missing charset for text
part {
    name "description"
    contentType "text/plain"  // Should include charset
    content "Description text"
}
```

55.4. Boundary Collisions

GroovyHttpClient automatically generates unique boundaries:

```
// Boundary format: ----GroovyBoundary{UUID}
// Example: ----GroovyBoundary3f8d9a1b-4e2c-4a5b-9f1e-8c7d3b2a1e0f
```

No manual boundary management required.

Chapter 56. Performance Considerations

Memory Usage	All parts loaded into memory — avoid very large files
Network	Multipart has overhead vs. direct binary POST
Retries	Entire payload retransmitted on retry
Timeouts	Scale timeout with total payload size

Chapter 57. Next Steps

Explore related topics:

- Circuit Breaker — Resilience for upload failures
- Request Configuration — Headers, retries, timeouts
- HttpClientTestDsl — Test multipart uploads easily
- Real-World Examples — Complete upload scenarios

Circuit Breaker

Chapter 58. Overview

The circuit breaker pattern prevents cascading failures by temporarily blocking requests to failing services. GroovyHttpClient integrates a production-ready circuit breaker that automatically monitors request failures and protects your application from overwhelming unhealthy dependencies.

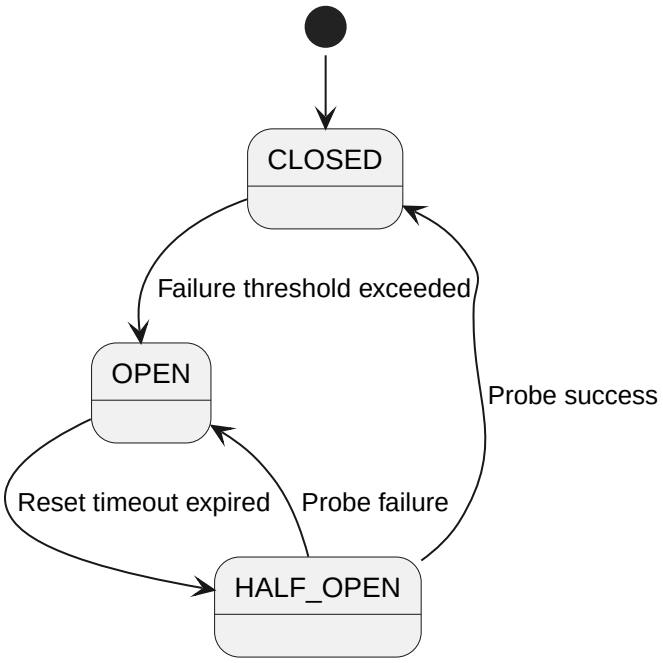
> Circuit breakers are like electrical breakers in your home — they prevent damage by cutting power when things go wrong. In distributed systems, they prevent cascading failures by temporarily blocking calls to unhealthy services.

58.1. Key Features

- □ Automatic failure detection and recovery
- □ Three states: CLOSED, OPEN, HALF_OPEN
- □ Configurable failure threshold
- □ Automatic reset after timeout
- □ Smart error classification (transient vs. permanent)
- □ Half-open probing for recovery detection
- □ Thread-safe operation
- □ Integration with retry policies

Chapter 59. Circuit Breaker States

The circuit breaker operates in three distinct states:



59.1. CLOSED State

Behavior	All requests pass through normally
Monitoring	Failure count increments on errors
Transition	Opens when failure count reaches threshold
Purpose	Normal operation mode

```
// Circuit is closed - all requests proceed
def response1 = client.getSync("/api/data")  // ✅ Executes
def response2 = client.getSync("/api/users") // ✅ Executes
def response3 = client.getSync("/api/posts") // ✅ Executes
```

59.2. OPEN State

Behavior	All requests fail immediately with `CircuitOpenException`
Monitoring	Tracks time since circuit opened
Transition	Moves to HALF_OPEN after reset timeout
Purpose	Protect failing service from additional load

```
// Circuit is open - requests fail fast
```

```
try {
    def response = client.getSync("/api/data")
} catch (GroovyHttpClient.CircuitOpenException e) {
    println "Circuit breaker is open - service unavailable"
    // Handle gracefully: return cached data, use fallback, etc.
}
```



The OPEN state prevents wasting resources on requests that will likely fail. Failing fast allows your application to:

- Show error messages immediately
- Return cached data
- Use fallback services
- Avoid thread pool exhaustion

59.3. HALF_OPEN State

Behavior	Single "probe" request allowed through
Monitoring	Probe result determines next state
Transition	CLOSED on success, OPEN on failure
Purpose	Test if service has recovered

```
// After reset timeout expires, circuit enters HALF_OPEN
// Next request acts as a probe

def response = client.getSync("/api/health")  // Probe request

// If successful → Circuit goes to CLOSED
// If fails → Circuit goes back to OPEN
```



Only ONE probe request is allowed in HALF_OPEN state. Subsequent requests while the probe is in-flight will fail with `CircuitOpenException`.

Chapter 60. Configuration

60.1. Via SecurityConfig

Configure circuit breaker when creating the client:

```
import org.softwood.http.SecurityConfig
import org.softwood.http.GroovyHttpClient

def config = SecurityConfig.production("https://api.example.com")
    .withOverrides([
        failureThreshold: 5,      // Open after 5 failures
        resetTimeoutMs: 30_000    // Try recovery after 30 seconds
    ])

def client = new GroovyHttpClient(config)
```

60.2. Via Builder

```
def client = GroovyHttpClient.builder("https://api.example.com")
    .failureThreshold(10)
    .resetTimeoutMs(60_000)  // 1 minute
    .build()
```

60.3. Configuration Parameters

Parameter	Description	Default
`failureThreshold`	Number of failures before opening circuit	5
`resetTimeoutMs`	Milliseconds before attempting recovery (HALF_OPEN)	30,000 (30 seconds)

Chapter 61. Failure Classification

The circuit breaker only counts **transient** or **server** errors. Not all errors trigger the circuit breaker.

61.1. Counted Failures (Circuit Breaker Opens)

```
// Network errors (transient)
 IOException
 HttpTimeoutException
 ConnectException

// Server errors (5xx)
 500 Internal Server Error
 502 Bad Gateway
 503 Service Unavailable
 504 Gateway Timeout
```

61.2. Ignored Failures (Circuit Stays Closed)

```
// Client errors (4xx) - These are not service health issues
 400 Bad Request
 401 Unauthorized
 403 Forbidden
 404 Not Found
 422 Unprocessable Entity

// Application errors - These are not transient
 IllegalArgumentException
 ContentParseException
 Custom business logic exceptions
```



This intelligent classification prevents the circuit breaker from opening due to client-side errors (bad requests, invalid auth) which wouldn't be fixed by waiting.

Chapter 62. Usage Patterns

62.1. Automatic Operation

The circuit breaker operates automatically — no explicit code needed:

```
// Create client with circuit breaker
def config = SecurityConfig.production("https://api.example.com")
    .withOverrides([failureThreshold: 3, resetTimeoutMs: 10_000])

def client = new GroovyHttpClient(config)

// Make requests normally
try {
    def response = client.getSync("/data")
    println "Success: ${response.body}"
} catch (GroovyHttpClient.CircuitOpenException e) {
    // Circuit is open - handle gracefully
    println "Service temporarily unavailable"
} catch (GroovyHttpClient.HttpResponseException e) {
    // Service returned error
    println "HTTP error: ${e.statusCode}"
} catch (IOException e) {
    // Network error (increments failure count)
    println "Network failure"
}
```

62.2. Graceful Degradation

Provide fallback behavior when circuit is open:

```
def fetchUserData(String userId) {
    try {
        return client.getSync("/users/${userId}").json
    } catch (GroovyHttpClient.CircuitOpenException e) {
        log.warn("Circuit breaker open - returning cached data")
        return cacheService.get("user:${userId}")
    }
}
```

62.3. Health Checks

Implement a health check endpoint that respects the circuit breaker:

```
def checkServiceHealth() {
```

```
    try {
        def response = client.getSync("/health")
        return [
            status: "healthy",
            circuitState: "CLOSED"
        ]
    } catch (GroovyHttpClient.CircuitOpenException e) {
        return [
            status: "degraded",
            circuitState: "OPEN",
            message: "Downstream service unavailable"
        ]
    }
}
```

62.4. Multiple Backends

Use separate circuit breakers for independent services:

```
// Primary service
def primaryClient = new GroovyHttpClient(
    SecurityConfig.production("https://primary-api.com")
        .withOverrides([failureThreshold: 5, resetTimeoutMs: 30_000])
)

// Backup service (more tolerance)
def backupClient = new GroovyHttpClient(
    SecurityConfig.production("https://backup-api.com")
        .withOverrides([failureThreshold: 10, resetTimeoutMs: 60_000])
)

def fetchData() {
    try {
        return primaryClient.getSync("/data").json
    } catch (GroovyHttpClient.CircuitOpenException e) {
        log.info("Primary circuit open - trying backup")
        return backupClient.getSync("/data").json
    }
}
```

Chapter 63. Integration with Retries

Circuit breakers work seamlessly with retry policies:

```
import org.softwood.http.GroovyHttpClient.RetryPolicy

def retryPolicy = new RetryPolicy.Config().tap {
    maxAttempts(3)
    backoffExponential(200, 2000, true)
    on5xx(true)
    onTimeouts(true)
}.build()

def config = SecurityConfig.production("https://api.example.com")
    .withOverrides([
        retryPolicy: retryPolicy,
        failureThreshold: 5,
        resetTimeoutMs: 30_000
    ])

def client = new GroovyHttpClient(config)
```



Execution flow with retries + circuit breaker:

1. Request attempt 1 → Fails (network error)
2. Retry after backoff → Fails (5xx error)
3. Retry after backoff → Fails (timeout)
4. All attempts exhausted → Circuit breaker counts as 1 failure
5. After 5 such failed request sequences → Circuit opens

The circuit breaker counts **request sequences**, not individual retry attempts.

Chapter 64. Advanced Patterns

64.1. Manual State Inspection

Access circuit breaker state programmatically:

```
// Access the circuit breaker
def breaker = client.circuitBreaker

// Check state
switch (breaker.state) {
    case GroovyHttpClient.State.CLOSED:
        println "Service healthy"
        break
    case GroovyHttpClient.State.OPEN:
        println "Service unavailable"
        break
    case GroovyHttpClient.State.HALF_OPEN:
        println "Service recovery in progress"
        break
}

// Get metrics
println "Failure count: ${breaker.failureCount}"
println "Failure threshold: ${breaker.failureThreshold}"
println "Reset timeout: ${breaker.resetTimeoutMs}ms"
```



The circuit breaker is designed for automatic operation. Manual state inspection should only be used for:

- Monitoring/metrics
- Health check endpoints
- Testing

Do NOT manually manipulate circuit breaker state in production code.

64.2. Metrics Collection

Track circuit breaker behavior:

```
import java.util.concurrent.atomic.AtomicLong

class CircuitBreakerMetrics {
    final AtomicLong openCount = new AtomicLong(0)
    final AtomicLong halfOpenCount = new AtomicLong(0)
    final AtomicLong successfulProbes = new AtomicLong(0)
```

```
    final AtomicLong failedProbes = new AtomicLong(0)

    void recordStateChange(state) {
        switch (state) {
            case GroovyHttpClient.State.OPEN:
                openCount.incrementAndGet()
                break
            case GroovyHttpClient.State.HALF_OPEN:
                halfOpenCount.incrementAndGet()
                break
        }
    }
}

// Monitor via interceptor
def metrics = new CircuitBreakerMetrics()

client.addInterceptor { ctx, next ->
    def breaker = client.circuitBreaker
    def initialState = breaker.state

    next.call(ctx).whenComplete { response, error ->
        if (breaker.state != initialState) {
            metrics.recordStateChange(breaker.state)
        }

        if (initialState == GroovyHttpClient.State.HALF_OPEN) {
            if (error == null) {
                metrics.successfulProbes.incrementAndGet()
            } else {
                metrics.failedProbes.incrementAndGet()
            }
        }
    }
}
```

64.3. Circuit Breaker Per Endpoint

For fine-grained control, create separate clients per critical endpoint:

```
class ApiClients {
    private final String baseUrl

    // Strict circuit breaker for critical endpoint
    final GroovyHttpClient criticalClient

    // Lenient circuit breaker for non-critical
    final GroovyHttpClient normalClient
```

```
    ApiClients(String baseUrl) {
        this.baseUrl = baseUrl

        this.criticalClient = new GroovyHttpClient(
            SecurityConfig.production(baseUrl)
                .withOverrides([failureThreshold: 2, resetTimeoutMs: 10_000])
        )

        this.normalClient = new GroovyHttpClient(
            SecurityConfig.production(baseUrl)
                .withOverrides([failureThreshold: 10, resetTimeoutMs: 60_000])
        )
    }

    def placeCriticalOrder(orderData) {
        criticalClient.postSync("/orders", null) { json(orderData) }
    }

    def fetchRecommendations() {
        normalClient.getSync("/recommendations")
    }
}
```

Chapter 65. Testing Circuit Breakers

65.1. With Spock

```
import spock.lang.Specification
import org.softwood.http.GroovyHttpClient
import org.softwood.http.SecurityConfig

class CircuitBreakerSpec extends Specification {

    def "circuit breaker opens after threshold failures"() {
        given: "a client with low threshold"
        def config = SecurityConfig.testing("http://failing-service.com")
            .withOverrides([
                failureThreshold: 3,
                resetTimeoutMs: 60_000
            ])
        def client = new GroovyHttpClient(config)

        and: "a server that returns 503"
        server.when("/data").respondWithStatus(503)

        when: "making requests until threshold"
        3.times {
            try {
                client.getSync("/data")
            } catch (ignored) {}
        }

        then: "circuit breaker opens"
        def error = null
        try {
            client.getSync("/data")
        } catch (GroovyHttpClient.CircuitOpenException e) {
            error = e
        }
        error != null
        error.message.contains("Circuit")
    }

    def "circuit breaker closes after reset timeout"() {
        given: "a client with short reset timeout"
        def config = SecurityConfig.testing("http://test-service.com")
            .withOverrides([
                failureThreshold: 2,
                resetTimeoutMs: 100  // 100ms
            ])
        def client = new GroovyHttpClient(config)
```

```
        when: "opening the circuit"
        server.when("/data").respondWithStatus(503)
        2.times {
            try { client.getSync("/data") } catch (ignored) {}
        }

        then: "circuit is open"
        def ex1 = null
        try {
            client.getSync("/data")
        } catch (GroovyHttpClient.CircuitOpenException e) {
            ex1 = e
        }
        ex1 != null

        when: "waiting for reset timeout"
        Thread.sleep(150)
        server.when("/data").respondWithStatus(200)

        and: "making a probe request"
        def response = client.getSync("/data")

        then: "probe succeeds and circuit closes"
        response.statusCode == 200
    }
}
```

65.2. Simulating Failures

```
def testDsl = HttpClientTestDsl.create()
testDsl.start()

// Simulate failing endpoint
testDsl.when("/flaky")
    .respondWith {
        status 503
        text "Service Unavailable"
    }

def client = new GroovyHttpClient(testDsl.baseUrl)
    .tap {
        config = SecurityConfig.testing(testDsl.baseUrl)
            .withOverrides([failureThreshold: 3, resetTimeoutMs: 10_000])
    }

// Trigger circuit breaker
3.times {
    try { client.getSync("/flaky") } catch (ignored) {}
}
```

```
// Verify circuit is open
try {
    client.getSync("/flaky")
    assert false, "Should have thrown CircuitOpenException"
} catch (GroovyHttpClient.CircuitOpenException e) {
    println "Circuit breaker working correctly"
}

testDsl.stop()
```

Chapter 66. Best Practices

66.1. Choose Appropriate Thresholds

Low Threshold (2-3)	Critical services, fast failure detection
Medium Threshold (5-10)	Normal services, balance resilience and tolerance
High Threshold (10-20)	Non-critical services, maximum tolerance

```
// ✅ Critical payment service
def paymentClient = new GroovyHttpClient(
    SecurityConfig.production("https://payment-api.com")
        .withOverrides([failureThreshold: 2, resetTimeoutMs: 10_000])
)

// ✅ Best-effort analytics service
def analyticsClient = new GroovyHttpClient(
    SecurityConfig.production("https://analytics-api.com")
        .withOverrides([failureThreshold: 20, resetTimeoutMs: 120_000])
)
```

66.2. Scale Reset Timeout with Recovery Time

Fast Recovery (10-30s)	Stateless services, quick deployments
Medium Recovery (30-60s)	Typical microservices
Slow Recovery (60-300s)	Databases, cache warm-up required

```
// ✅ Stateless API
resetTimeoutMs: 10_000  // 10 seconds

// ✅ Service with cache warm-up
resetTimeoutMs: 60_000  // 1 minute

// ✅ Database recovering from backup
resetTimeoutMs: 300_000 // 5 minutes
```

66.3. Combine with Retries Carefully

```
// ✅ Good: Few retries, moderate circuit breaker
def retryPolicy = new RetryPolicy.Config().tap {
    maxAttempts(3)
    backoffExponential(200, 1000, true)
```

```
}.build()

def config = SecurityConfig.production(url)
    .withOverrides([
        retryPolicy: retryPolicy,
        failureThreshold: 5,
        resetTimeoutMs: 30_000
    ])

// ❌ Bad: Many retries + strict circuit breaker
// Circuit will open too quickly
maxAttempts(10)  // ← Too many retries
failureThreshold: 2  // ← Too strict threshold
```

66.4. Always Handle CircuitOpenException

```
// ✅ Proper error handling
try {
    return client.getSync("/data").json
} catch (GroovyHttpClient.CircuitOpenException e) {
    log.warn("Circuit open - using fallback")
    return fallbackData
} catch (Exception e) {
    log.error("Request failed", e)
    throw e
}

// ❌ No circuit breaker handling
try {
    return client.getSync("/data").json
} catch (Exception e) {
    // Circuit open exception not handled specially
    throw e
}
```

66.5. Monitor Circuit Breaker State

```
// ✅ Log state transitions
client.addInterceptor { ctx, next ->
    def initialState = client.circuitBreaker.state

    next.call(ctx).whenComplete { response, error ->
        def newState = client.circuitBreaker.state
        if (initialState != newState) {
            log.warn("Circuit breaker state changed: ${initialState} → ${newState}")
        }
    }
```

```
}
```

66.6. Don’t Disable for Testing

```
// ❌ Bad: Disabled circuit breaker
def config = SecurityConfig.testing(url)
    .withOverrides([failureThreshold: Integer.MAX_VALUE])

// ✅ Good: Use realistic thresholds even in tests
def config = SecurityConfig.testing(url)
    .withOverrides([failureThreshold: 5, resetTimeoutMs: 1000])
```

Chapter 67. Troubleshooting

67.1. Circuit Opens Too Quickly

```
// Problem: Circuit opens after just 2 failures
failureThreshold: 2

// Solution: Increase threshold
failureThreshold: 10

// Or: Add retries to absorb transient failures
def retryPolicy = new RetryPolicy.Config().tap {
    maxAttempts(3)
    backoffExponential(200, 1000, true)
}.build()
```

67.2. Circuit Never Opens

```
// Problem: Only non-transient errors occurring
// 404 Not Found → Not counted
// 401 Unauthorized → Not counted

// Solution: Check that server is returning 5xx or
// ensure network/timeout errors are occurring

// Verify error types:
client.addInterceptor { ctx, next ->
    next.call(ctx).exceptionally { error ->
        println "Error type: ${error.class.name}"
        println "Status code: ${error instanceof HttpResponseException ?
error.statusCode : 'N/A'}"
        throw error
    }
}
```

67.3. Circuit Stays Open Forever

```
// Problem: Reset timeout too long
resetTimeoutMs: 3_600_000  // 1 hour

// Solution: Use reasonable timeout
resetTimeoutMs: 30_000  // 30 seconds

// Or: Check that service is actually recovering
```

67.4. Probe Request Fails

```
// Circuit moves OPEN → HALF_OPEN → OPEN (immediately)

// Possible causes:
// 1. Service hasn't fully recovered
// 2. Probe endpoint is unhealthy
// 3. Reset timeout too short

// Solutions:
// 1. Increase reset timeout to allow more recovery time
// 2. Use a dedicated health endpoint for probes
// 3. Investigate service recovery process
```

Chapter 68. Performance Impact

Memory	Minimal (single counter + timestamps per client)
CPU	Negligible (simple integer comparisons)
Latency	~1µs overhead per request
Concurrency	Thread-safe with atomic operations

Chapter 69. Next Steps

Related topics for robust HTTP clients:

- Request Configuration — Timeouts and retries
- Multipart Uploads — File upload resilience
- HttpClientTestDsl — Test circuit breaker behavior
- Real-World Examples — Production patterns

Testing with MockHttpServer

Chapter 70. Overview

`MockHttpServer` is a lightweight, dependency-free HTTP test server built on Java's `com.sun.net.httpserver.HttpServer`. It provides comprehensive request validation capabilities perfect for testing HTTP clients without external dependencies or complex setup.

> Testing HTTP clients shouldn't require Docker containers or external services. MockHttpServer makes testing fast, reliable, and simple.

70.1. Key Features

- Zero external dependencies (uses only JDK)
- Automatic port allocation (no conflicts)
- Request validation (headers, cookies, multipart)
- Flexible response configuration
- Latency simulation
- Thread-safe for concurrent tests
- Spock and JUnit 5 compatible

70.2. When to Use MockHttpServer

Unit Tests	Verify client behavior without network calls
Integration Tests	Test HTTP interactions in isolation
Multipart Testing	Validate file upload payloads
Error Simulation	Test retry logic, timeouts, error handling
Cookie Testing	Verify cookie flow across requests

Chapter 71. Quick Start

71.1. Basic Server Setup

```
import org.softwood.test.MockHttpServer

// Create server
def server = new MockHttpServer()
server.init()  // Starts on random port

try {
    // Configure endpoints
    server.addRequestCheck("GET", "/hello", 200)
        .withResponseBody("Hello, World!")

    // Create client pointing to server
    def client = new GroovyHttpClient("http://localhost:${server.port}")

    // Make request
    def response = client.getSync("/hello")

    assert response.statusCode == 200
    assert response.body == "Hello, World!"

} finally {
    server.shutdown()
}
```

71.2. With Groovy Try-With-Resources

For automatic cleanup:

```
def server = new MockHttpServer()
server.init()

try {
    server.addRequestCheck("GET", "/data", 200)
        .withResponseBody('{"status":"ok"}')

    def client = new GroovyHttpClient("http://localhost:${server.port}")
    def response = client.getSync("/data")

    assert response.json.status == "ok"
} finally {
    server.shutdown()
}
```

Chapter 72. Request Validation

72.1. Method Validation

Ensure requests use the correct HTTP method:

```
// Only accept POST
server.addRequestCheck("POST", "/users", 201)
    .withResponseBody('{"id":123}')

// GET returns 405 Method Not Allowed
def response = client.getSync("/users")
assert response.statusCode == 405
```

72.2. Header Validation

Verify required headers are present:

```
server.addRequestCheck("POST", "/api/data", 201)
    .withExpectedHeaders([
        "Content-Type": ["application/json"],
        "X-API-Key": ["secret123"]
    ])
    .withResponseBody('{"created":true}')

// Request without headers → 400
def response1 = client.postSync("/api/data", "{}")
assert response1.statusCode == 400

// Request with headers → 201
def response2 = client.postSync("/api/data", "{}") {
    header "Content-Type", "application/json"
    header "X-API-Key", "secret123"
}
assert response2.statusCode == 201
```



Header matching is **case-insensitive** and supports multiple values per header.

72.3. Cookie Validation

Validate cookies sent by the client:

```
server.addRequestCheck("GET", "/profile", 200)
    .withExpectedCookies([
        session: "abc123",
```

```
        user_id: "42"
    ])
    .withResponseBody('{"name":"Alice"}')

// Add cookies to client
client.addCookie("session", "abc123")
client.addCookie("user_id", "42")

def response = client.getSync("/profile")
assert response.statusCode == 200
```

Missing or mismatched cookies return 400:

```
// Wrong cookie value
client.addCookie("session", "wrong")
def response = client.getSync("/profile")
assert response.statusCode == 400
```

72.4. Query Parameter Validation

While MockHttpServer doesn't have built-in query parameter validation, you can validate via custom checks:

```
server.addRequestCheck("GET", "/search", 200)
    .withResponseBody('{"results":[]}')

def response = client.getSync("/search") {
    query(q: "groovy", limit: 10)
}

// Manually verify query params if needed
assert response.statusCode == 200
```

Chapter 73. Response Configuration

73.1. Status Codes

Configure response status:

```
// Success
server.addRequestCheck("GET", "/ok", 200)

// Created
server.addRequestCheck("POST", "/users", 201)

// No Content
server.addRequestCheck("DELETE", "/users/123", 204)

// Not Found
server.addRequestCheck("GET", "/missing", 404)

// Server Error
server.addRequestCheck("GET", "/error", 500)
```

73.2. Response Bodies

Plain Text:

```
server.addRequestCheck("GET", "/text", 200)
    .withResponseBody("Hello, World!")
```

JSON:

```
import groovy.json.JsonOutput

def jsonData = [
    id: 123,
    name: "Alice",
    roles: ["admin", "user"]
]

server.addRequestCheck("GET", "/user", 200)
    .withResponseBody(JsonOutput.toJson(jsonData))
    .withResponseHeaders(["Content-Type": ["application/json"]])
```

Empty Body:

```
server.addRequestCheck("DELETE", "/users/123", 204)
```

```
    .withResponseBody("")  // or omit entirely
```

73.3. Response Headers

Single Header:

```
server.addRequestCheck("GET", "/data", 200)
    .withResponseBody('{"data":"value"}')
    .withResponseHeaders([
        "Content-Type": ["application/json"]
    ])
```

Multiple Headers:

```
server.addRequestCheck("GET", "/resource", 200)
    .withResponseHeaders([
        "Content-Type": ["application/json"],
        "Cache-Control": ["no-cache", "no-store"],
        "X-RateLimit-Remaining": ["42"]
    ])
```



Always set `Content-Type` for non-text responses to ensure proper parsing by clients.

73.4. Response Cookies

Set cookies in the response:

```
server.addRequestCheck("POST", "/login", 200)
    .withResponseBody('{"success":true}')
    .withResponseCookies([
        session: "abc123",
        user_id: "42"
    ])

def response = client.postSync("/login", null) {
    form(username: "alice", password: "secret")
}

// Cookies automatically stored by client
assert client.getCookie("session").value == "abc123"
```

Chapter 74. Simulating Latency

Add artificial delays to responses:

```
// Slow endpoint (1 second delay)
server.addRequestCheck("GET", "/slow", 200)
    .withResponseBody('{"data":"delayed"}')
    .withDelay(1000)

def start = System.currentTimeMillis()
def response = client.getSync("/slow")
def elapsed = System.currentTimeMillis() - start

assert elapsed >= 1000
assert response.statusCode == 200
```

Use cases for delays:

- Testing timeout behavior
- Simulating slow APIs
- Testing retry logic
- Load testing with realistic latency

```
// Test timeout handling
server.addRequestCheck("GET", "/timeout", 200)
    .withDelay(5000)  // 5 second delay

def config = SecurityConfig.testing("http://localhost:${server.port}")
    .withOverrides([requestTimeout: Duration.ofSeconds(2)])

def client = new GroovyHttpClient(config)

try {
    client.getSync("/timeout")
    assert false, "Should have timed out"
} catch (HttpTimeoutException e) {
    // Expected
}
```

Chapter 75. Error Simulation

75.1. Client Errors (4xx)

```
// Bad Request
server.addRequestCheck("POST", "/users", 400)
    .withResponseBody('{"error":"Invalid email"}')

// Unauthorized
server.addRequestCheck("GET", "/admin", 401)
    .withResponseBody('{"error":"Authentication required"}')

// Forbidden
server.addRequestCheck("DELETE", "/system", 403)
    .withResponseBody('{"error":"Insufficient permissions"}')

// Not Found
server.addRequestCheck("GET", "/missing", 404)
    .withResponseBody('{"error":"Resource not found"}')
```

75.2. Server Errors (5xx)

```
// Internal Server Error
server.addRequestCheck("GET", "/buggy", 500)
    .withResponseBody('{"error":"Internal server error"}')

// Bad Gateway
server.addRequestCheck("GET", "/proxy", 502)
    .withResponseBody('{"error":"Upstream service unavailable"}')

// Service Unavailable
server.addRequestCheck("GET", "/maintenance", 503)
    .withResponseBody('{"error":"Service under maintenance"}')

// Gateway Timeout
server.addRequestCheck("GET", "/slow-upstream", 504)
    .withResponseBody('{"error":"Upstream timeout"}')
```

75.3. Testing Retry Logic

```
import java.util.concurrent.atomic.AtomicInteger

def attemptCount = new AtomicInteger(0)

// Fail first 2 attempts, succeed on 3rd
```

```
server.addRequestCheck("GET", "/flaky", 200)
    .withResponseBody('{"success":true}')

// Client with retry policy
def retryPolicy = new RetryPolicy.Config().tap {
    maxAttempts(3)
    backoffExponential(100, 1000, false)
    on5xx(true)
}.build()

def config = SecurityConfig.testing("http://localhost:${server.port}")
    .withOverrides([retryPolicy: retryPolicy])

def client = new GroovyHttpClient(config)

// Add interceptor to track attempts
client.addInterceptor { ctx, next ->
    attemptCount.incrementAndGet()
    next.call(ctx)
}

def response = client.getSync("/flaky")
assert attemptCount.get() >= 1  // At least one attempt made
```

Chapter 76. Multipart Request Testing

MockHttpServer can validate multipart/form-data uploads:

```
server.addRequestCheck("POST", "/upload", 200)
    .withExpectedMultipart([
        [name: "file", filename: "test.txt", content: "Hello!".bytes],
        [name: "description", content: "Test file".bytes]
    ])
    .withResponseBody('{"uploaded":true}')

// Client upload
def response = client.postMultipartSync("/upload") {
    part {
        name "file"
        filename "test.txt"
        content "Hello!"
    }
    part {
        name "description"
        content "Test file"
    }
}

assert response.statusCode == 200
```



Multipart validation checks:

- Part names match
- Filenames match (if specified)
- Content-Types match (if specified)
- Binary content matches byte-for-byte

Chapter 77. Spock Integration

MockHttpServer integrates seamlessly with Spock:

```
import spock.lang.Specification
import spock.lang.AutoCleanup

class UserApiSpec extends Specification {

    @AutoCleanup
    MockHttpServer server = new MockHttpServer()

    GroovyHttpClient client

    def setup() {
        server.init()
        client = new GroovyHttpClient("http://localhost:${server.port}")
    }

    def "should fetch user by ID"() {
        given: "a mocked user endpoint"
        server.addRequestCheck("GET", "/users/123", 200)
            .withResponseBody('{"id":123,"name":"Alice"}')
            .withResponseHeaders(["Content-Type": ["application/json"]])

        when: "fetching user"
        def response = client.getSync("/users/123")
        def user = response.json

        then: "user data is correct"
        response.statusCode == 200
        user.id == 123
        user.name == "Alice"
    }

    def "should handle missing users"() {
        given: "a 404 response"
        server.addRequestCheck("GET", "/users/999", 404)
            .withResponseBody('{"error":"Not found"}')

        when: "fetching non-existent user"
        client.getSync("/users/999")

        then: "throws exception"
        def ex = thrown(HttpResponseException)
        ex.statusCode == 404
    }

    def "should validate authentication headers"() {
        given: "an endpoint requiring auth"
```

```
        server.addRequestCheck("GET", "/admin", 200)
            .withExpectedHeaders(["Authorization": ["Bearer secret"]])
            .withResponseBody('{"data":"sensitive"}')

        and: "client with auth header"
        client.setHeader("Authorization", "Bearer secret")

        when: "making authenticated request"
        def response = client.getSync("/admin")

        then: "request succeeds"
        response.statusCode == 200
    }

    def "should handle rate limiting"() {
        given: "a rate-limited endpoint"
        server.addRequestCheck("GET", "/api/data", 429)
            .withResponseBody('{"error":"Rate limit exceeded"}')
            .withResponseHeaders(["Retry-After": ["60"]])

        when: "exceeding rate limit"
        def response = client.getSync("/api/data")

        then: "returns 429"
        def ex = thrown(HttpResponseException)
        ex.statusCode == 429
    }
}
```

77.1. Setup/Cleanup Patterns

Per-Test Cleanup:

```
class ApiSpec extends Specification {
    MockHttpServer server

    def setup() {
        server = new MockHttpServer()
        server.init()
    }

    def cleanup() {
        server?.shutdown()
    }
}
```

Shared Server (Faster):

```
class ApiSpec extends Specification {
    @Shared
    MockHttpServer server = new MockHttpServer()

    def setupSpec() {
        server.init()
    }

    def cleanupSpec() {
        server.shutdown()
    }

    def cleanup() {
        // Reset endpoints between tests
        server.checks.clear()
    }
}
```

Chapter 78. JUnit 5 Integration

```
import org.junit.jupiter.api.*

class UserApiTest {
    private MockHttpServer server
    private GroovyHttpClient client

    @BeforeEach
    void setup() {
        server = new MockHttpServer()
        server.init()
        client = new GroovyHttpClient("http://localhost:${server.port}")
    }

    @AfterEach
    void teardown() {
        server.shutdown()
    }

    @Test
    void shouldFetchUser() {
        // given
        server.addRequestCheck("GET", "/users/123", 200)
            .withResponseBody('{"id":123,"name":"Alice"}')

        // when
        def response = client.getSync("/users/123")

        // then
        Assertions.assertEquals(200, response.statusCode)
        Assertions.assertEquals("Alice", response.json.name)
    }
}
```

Chapter 79. Advanced Patterns

79.1. Dynamic Response Generation

```
def userDb = [
    "123": [id: 123, name: "Alice"],
    "456": [id: 456, name: "Bob"]
]

// Note: MockHttpServer doesn't support dynamic responses directly
// For complex scenarios, use HttpClientTestDsl instead
```

79.2. Multiple Endpoints

```
server.addRequestCheck("GET", "/users", 200)
    .withResponseBody('[{"id":1},{"id":2}]')
    .withResponseHeaders(["Content-Type": ["application/json"]])

server.addRequestCheck("GET", "/users/123", 200)
    .withResponseBody('{"id":123,"name":"Alice"}')
    .withResponseHeaders(["Content-Type": ["application/json"]])

server.addRequestCheck("POST", "/users", 201)
    .withExpectedHeaders(["Content-Type": ["application/json"]])
    .withResponseBody('{"id":789}')
    .withResponseHeaders(["Content-Type": ["application/json"]])

server.addRequestCheck("DELETE", "/users/123", 204)
```

79.3. Testing Redirect Handling

```
// MockHttpServer doesn't directly support redirects
// Test redirect handling by:
// 1. Returning 301/302 with Location header
// 2. Verifying client follows (or doesn't follow) redirect

server.addRequestCheck("GET", "/old-url", 301)
    .withResponseHeaders(["Location": ["http://localhost:${server.port}/new-url"]])
    .withResponseBody("")

server.addRequestCheck("GET", "/new-url", 200)
    .withResponseBody('{"redirected":true}')

// Client with redirects enabled
def config = SecurityConfig.testing("http://localhost:${server.port}")
```

```
    .withOverrides([allowRedirects: true])

def client = new GroovyHttpClient(config)
def response = client.getSync("/old-url")
// Response should be from /new-url
```

79.4. Cookie Flow Testing

```
// Login sets cookie
server.addRequestCheck("POST", "/login", 200)
    .withResponseCookies([session: "abc123"])
    .withResponseBody('{"success":true}')

// Profile requires cookie
server.addRequestCheck("GET", "/profile", 200)
    .withExpectedCookies([session: "abc123"])
    .withResponseBody('{"name":"Alice"}')

// Logout clears cookie
server.addRequestCheck("POST", "/logout", 200)
    .withExpectedCookies([session: "abc123"])
    .withResponseBody('{"success":true}')

// Test flow
def loginResp = client.postSync("/login", null) {
    form(username: "alice", password: "secret")
}
assert loginResp.statusCode == 200

// Cookie automatically included
def profileResp = client.getSync("/profile")
assert profileResp.statusCode == 200

def logoutResp = client.postSync("/logout", "")
assert logoutResp.statusCode == 200
```

Chapter 80. Best Practices

80.1. Use Descriptive Endpoints

```
// ✅ Good: Descriptive, intention-revealing
server.addRequestCheck("GET", "/users/123", 200)
    .withResponseBody('{"id":123,"name":"Alice"}')

server.addRequestCheck("POST", "/users/activate", 200)
    .withExpectedHeaders(["X-Admin-Token": ["secret"]])

// ❌ Avoid: Generic, unclear
server.addRequestCheck("GET", "/test", 200)
server.addRequestCheck("POST", "/endpoint", 201)
```

80.2. Organize Tests by Scenario

```
class UserApiSpec extends Specification {
    MockHttpServer server
    GroovyHttpClient client

    def setup() {
        server = new MockHttpServer()
        server.init()
        client = new GroovyHttpClient("http://localhost:${server.port}")
    }

    def cleanup() {
        server.shutdown()
    }

    def "GET /users - success"() { /* ... */ }
    def "GET /users - unauthorized"() { /* ... */ }
    def "GET /users - server error"() { /* ... */ }

    def "POST /users - create success"() { /* ... */ }
    def "POST /users - validation error"() { /* ... */ }

    def "DELETE /users/123 - success"() { /* ... */ }
    def "DELETE /users/123 - not found"() { /* ... */ }
}
```

80.3. Reset State Between Tests

```
def cleanup() {
```

```
    server.checks.clear()  // Clear registered endpoints
}
```

80.4. Use Realistic Response Data

```
// ✅ Realistic
def userData = [
    id: 123,
    name: "Alice Smith",
    email: "alice@example.com",
    created_at: "2024-01-15T10:30:00Z",
    roles: ["user", "admin"]
]

// ❌ Minimal (harder to catch bugs)
def userData = [id: 1, name: "A"]
```

80.5. Test Error Scenarios

Don't just test happy paths:

```
def "should handle 400 Bad Request"() { /* ... */ }
def "should handle 401 Unauthorized"() { /* ... */ }
def "should handle 404 Not Found"() { /* ... */ }
def "should handle 500 Internal Server Error"() { /* ... */ }
def "should handle 503 Service Unavailable"() { /* ... */ }
def "should retry on timeout"() { /* ... */ }
def "should open circuit breaker on failures"() { /* ... */ }
```

Chapter 81. Limitations

MockHttpServer is designed for straightforward request/response testing. For more complex scenarios, consider using `HttpClientTestDsl`:

Not Supported	Dynamic responses based on request content
Not Supported	Request body validation (beyond multipart)
Not Supported	WebSocket connections
Not Supported	Server-sent events (SSE)
Not Supported	Chunked transfer encoding

For these scenarios, use HttpClientTestDsl which provides more flexibility.

Chapter 82. Troubleshooting

82.1. Port Already in Use

```
// Solution: Let server choose port automatically
def server = new MockHttpServer()  // Port 0 = auto-assign
server.init()
println "Server running on port: ${server.port}"
```

82.2. Headers Not Matching

```
// Remember: Header names are case-insensitive
// These are equivalent:
.withExpectedHeaders(["content-type": ["application/json"]])
.withExpectedHeaders(["Content-Type": ["application/json"]])
.withExpectedHeaders(["CONTENT-TYPE": ["application/json"]])
```

82.3. Cookies Not Matching

```
// Cookies are matched exactly (case-sensitive values)
server.addRequestCheck("GET", "/test", 200)
    .withExpectedCookies([session: "ABC123"])

// This fails:
client.addCookie("session", "abc123")  // Wrong case

// This succeeds:
client.addCookie("session", "ABC123")  // Exact match
```

82.4. Server Not Starting

```
// Always call init() before use
def server = new MockHttpServer()
server.init()  // Required!

// Don't forget cleanup
try {
    // Tests...
} finally {
    server.shutdown()  // Always cleanup
}
```

Chapter 83. Next Steps

Now that you understand MockHttpServer, learn about more advanced testing tools:

- HttpClientTestDsl — Fluent DSL for flexible testing
- HttpClientTestTrait — Spock trait for seamless integration
- Real-World Testing Examples — Complete test suite examples

Testing DSL (HttpClientTestDsl)

Chapter 84. Overview

`HttpClientTestDsl` is a fluent, embedded HTTP test server specifically designed for testing GroovyHttpClient interactions. Unlike MockHttpServer's validation-focused approach, HttpClientTestDsl provides a flexible, DSL-driven API for defining mock endpoints, recording requests, and verifying interactions.

> Testing HTTP clients should be as expressive as the code you're testing. HttpClientTestDsl makes writing tests feel like writing executable specifications.

84.1. Key Features

- Embedded HTTP server with automatic port allocation
- Fluent DSL for defining mock endpoints
- Request recording and history tracking
- Rich verification API for assertions
- JSON and XML response builders
- Custom response headers and status codes
- Request matching by method, headers, query params, body
- Multipart request testing
- Thread-safe for concurrent tests
- Spock and JUnit 5 integration

84.2. When to Use HttpClientTestDsl

Complex Tests	Multiple endpoints with varied responses
Request Verification	Assert on request headers, body, params
Dynamic Responses	Different responses per call
Request History	Verify call counts, order, content
Flexible Mocking	API-style testing over strict validation

Chapter 85. Quick Start

85.1. Basic Server Setup

```
import org.softwood.http.HttpClientTestDsl
import org.softwood.http.GroovyHttpClient

// Create and start test server
def testDsl = HttpClientTestDsl.create()
testDsl.start()

// Define mock endpoint
testDsl.when("/api/users")
    .respondWith {
        status 200
        json([
            users: [
                [id: 1, name: "Alice"],
                [id: 2, name: "Bob"]
            ]
        ])
    }

// Create client pointing to test server
def client = testDsl.createClient()

// Make request
def response = client.getSync("/api/users")

// Assertions
assert response.statusCode == 200
assert response.json.users.size() == 2
assert response.json.users[0].name == "Alice"

// Verify request was made
testDsl.verify("/api/users") {
    wasCalled(1)
    withMethod("GET")
}

// Cleanup
testDsl.stop()
```



Use `testDsl.createClient()` instead of manually constructing a GroovyHttpClient. This automatically configures the client with:

- Correct base URL

- Testing security profile
- Appropriate timeouts

85.2. With Try-With-Resources

For automatic cleanup:

```
HttpClientTestDsl.create().withCloseable { testDsl ->
    testDsl.start()

    testDsl.when("/data")
        .respondWith {
            status 200
            text "Success"
        }

    def client = testDsl.createClient()
    def response = client.getSync("/data")

    assert response.statusCode == 200
}
// Automatically stops server
```

Chapter 86. Defining Mock Endpoints

86.1. Basic Endpoint

```
testDsl.when("/api/products")
    .respondWith {
        status 200
        json([
            products: [
                [id: 1, name: "Laptop", price: 999.99],
                [id: 2, name: "Mouse", price: 29.99]
            ]
        ])
    }
```

86.2. Multiple Endpoints

Define multiple endpoints for different paths:

```
// List users
testDsl.when("/api/users")
    .respondWith {
        status 200
        json([users: []])
    }

// Get user by ID
testDsl.when("/api/users/123")
    .respondWith {
        status 200
        json([id: 123, name: "Alice", email: "alice@example.com"])
    }

// Create user
testDsl.when("/api/users")
    .respondWith {
        status 201
        json([id: 456, message: "User created"])
    }

// Delete user
testDsl.when("/api/users/123")
    .respondWith {
        status 204
        body ""
    }
```



The same path can be defined multiple times. Each definition overrides the previous one. This is useful for test-specific setup:

```
def setup() {
    testDsl.when("/data").respondWith { status 200; text "default" }
}

def "specific test"() {
    // Override for this test
    testDsl.when("/data").respondWith { status 404; text "not found" }
}
```

86.3. Shorthand Methods

For simple responses:

```
// Status only
testDsl.when("/ping").respondWithStatus(200)

// JSON with default 200 status
testDsl.when("/data").respondWithJson([value: 42])

// JSON with custom status
testDsl.when("/error").respondWithJson([error: "Not found"], 404)
```

Chapter 87. Response Configuration

87.1. Status Codes

```
// Success
testDsl.when("/success").respondWith { status 200 }

// Created
testDsl.when("/created").respondWith { status 201 }

// No Content
testDsl.when("/deleted").respondWith { status 204 }

// Not Found
testDsl.when("/missing").respondWith { status 404 }

// Server Error
testDsl.when("/error").respondWith { status 500 }
```

87.2. Response Bodies

Plain Text:

```
testDsl.when("/message").respondWith {
    status 200
    text "Hello, World!"
}

// Or using body method
testDsl.when("/data").respondWith {
    status 200
    contentType "text/plain"
    body "Custom content"
}
```

JSON:

```
testDsl.when("/users/123").respondWith {
    status 200
    json([
        id: 123,
        name: "Alice Smith",
        email: "alice@example.com",
        roles: ["admin", "user"],
        metadata: [
            created: "2025-01-24T10:30:00Z",
```

```
            lastLogin: "2025-01-24T15:45:00Z"
        ]
    ])
}
```

XML:

```
def xmlContent = '''
<user>
    <id>123</id>
    <name>Alice</name>
    <email>alice@example.com</email>
</user>
'''

testDsl.when("/users/123.xml").respondWith {
    status 200
    xml xmlContent
}
```

HTML:

```
testDsl.when("/page").respondWith {
    status 200
    html """
        <html>
            <head><title>Test Page</title></head>
            <body><h1>Hello!</h1></body>
        </html>
    """
}
```

Empty Body:

```
testDsl.when("/deleted").respondWith {
    status 204
    body ""
}
```

87.3. Response Headers

Single Header:

```
testDsl.when("/data").respondWith {
    status 200
    header "X-Request-ID", UUID.randomUUID().toString()
```

```
    json([data: "value"])
}
```

Multiple Headers:

```
testDsl.when("/api/data").respondWith {
    status 200
    header "Content-Type", "application/json"
    header "Cache-Control", "no-cache"
    header "X-RateLimit-Remaining", "99"
    header "X-RateLimit-Reset", "3600"
    json([data: "value"])
}
```



Headers set via `header` method take precedence over content-type set by helper methods (`json`, `xml`, `text`, `html`).

87.4. Custom Content Types

```
testDsl.when("/data").respondWith {
    status 200
    contentType "application/vnd.api+json"
    body '{"data":{"type":"articles","id":"1"}}'
}
```

Chapter 88. Request Recording and Verification

88.1. Basic Verification

```
// Make request
client.getSync("/api/users")

// Verify it was called
testDsl.verify("/api/users") {
    wasCalled()  // At least once
}

// Verify call count
testDsl.verify("/api/users") {
    wasCalled(1)  // Exactly once
}

// Verify not called
testDsl.verify("/api/other") {
    wasNotCalled()
}
```

88.2. Method Verification

```
client.postSync("/api/users", null) {
    json([name: "Alice", email: "alice@example.com"])
}

testDsl.verify("/api/users") {
    wasCalled(1)
    withMethod("POST")
}
```

88.3. Header Verification

Check Header Exists:

```
client.getSync("/data") {
    header "Authorization", "Bearer token123"
}

testDsl.verify("/data") {
    wasCalled()
```

```
    withHeader("Authorization")
}
```

Check Header Value:

```
testDsl.verify("/data") {
    withHeader("Authorization", "Bearer token123")
}
```

Multiple Headers:

```
testDsl.verify("/data") {
    withHeader("Authorization", "Bearer token123")
    withHeader("Content-Type", "application/json")
    withHeader("X-Request-ID")
}
```

88.4. Body Verification

Exact Match:

```
client.postSync("/submit", "exact body content")

testDsl.verify("/submit") {
    withBody("exact body content")
}
```

Substring Match:

```
client.postSync("/submit", '{"name":"Alice","age":30}')

testDsl.verify("/submit") {
    withBodyContaining("Alice")
    withBodyContaining('"age":30')
}
```

JSON Body Matching:

```
client.postSync("/users", null) {
    json([name: "Alice", age: 30, email: "alice@example.com"])
}

testDsl.verify("/users") {
    withJsonBody { json ->
```

```
        json.name == "Alice" && json.age == 30
    }
}
```



The `withJsonBody` closure receives the parsed JSON and should return `true` for a match. This is more flexible than exact string matching.

88.5. Query Parameter Verification

Check Parameter Exists:

```
client.getSync("/search") {
    query(q: "groovy", limit: 10)
}

testDsl.verify("/search") {
    withQueryParam("q")
    withQueryParam("limit")
}
```

Check Parameter Value:

```
testDsl.verify("/search") {
    withQueryParam("q", "groovy")
    withQueryParam("limit", "10")
}
```

88.6. Accessing Recorded Requests

Get All Requests:

```
def allRequests = testDsl.recordedRequests
assert allRequests.size() == 3
```

Get Requests for Path:

```
def userRequests = testDsl.getRequestsFor("/api/users")
assert userRequests.size() == 2
```

Get First/Last Request:

```
testDsl.verify("/data") {
    wasCalled()
```

```
    def firstRequest = first()
    assert firstRequest.method == "GET"

    def lastRequest = last()
    assert lastRequest.hasHeader("Authorization")
}
```

88.7. RecordedRequest Properties

```
def request = testDsl.getRequestsFor("/data").first()

// Properties
String method = request.method                    // "GET", "POST", etc.
String path = request.path                        // "/data"
Map<String, List<String>> headers = request.headers
String body = request.body
Map<String, List<String>> queryParams = request.queryParams

// Helper methods
String header = request.getHeader("Content-Type")
boolean hasHeader = request.hasHeader("Authorization")
Object json = request.jsonBody
String param = request.getQueryParam("q")
List<String> params = request.getQueryParams("tags")
boolean hasParam = request.hasQueryParam("limit")
```

Chapter 89. Spock Integration

HttpClientTestDsl integrates seamlessly with Spock:

89.1. Setup/Cleanup Pattern

```
import spock.lang.Specification

class UserApiSpec extends Specification {

    HttpClientTestDsl testDsl
    GroovyHttpClient client

    def setup() {
        testDsl = HttpClientTestDsl.create()
        testDsl.start()
        client = testDsl.createClient()
    }

    def cleanup() {
        testDsl.stop()
    }

    def "should fetch user by ID"() {
        given: "a mock user endpoint"
        testDsl.when("/users/123").respondWith {
            status 200
            json([id: 123, name: "Alice", email: "alice@example.com"])
        }

        when: "fetching user"
        def response = client.getSync("/users/123")

        then: "user data is returned"
        response.statusCode == 200
        response.json.id == 123
        response.json.name == "Alice"

        and: "request was made"
        testDsl.verify("/users/123") {
            wasCalled(1)
            withMethod("GET")
        }
    }
}
```

89.2. Shared Server Pattern

For faster tests, share the server across test methods:

```
import spock.lang.Shared
import spock.lang.Specification

class UserApiSpec extends Specification {

    @Shared HttpClientTestDsl testDsl

    def setupSpec() {
        testDsl = HttpClientTestDsl.create()
        testDsl.start()
    }

    def cleanupSpec() {
        testDsl.stop()
    }

    def cleanup() {
        testDsl.reset()  // Clear endpoints and history between tests
    }

    def "test 1"() {
        given:
        testDsl.when("/data").respondWithJson([value: 1])
        // ... test code
    }

    def "test 2"() {
        given:
        testDsl.when("/data").respondWithJson([value: 2])
        // ... test code
    }
}
```

89.3. Complex Verification Example

```
def "should create user with proper authentication"() {
    given: "an authenticated endpoint"
    testDsl.when("/users").respondWith {
        status 201
        json([id: 456, message: "User created"])
    }

    and: "a client with auth header"
    client.setHeader("Authorization", "Bearer token123")
```

```
    when: "creating a user"
    def userData = [
        name: "Bob",
        email: "bob@example.com",
        roles: ["user"]
    ]
    def response = client.postSync("/users", null) {
        json(userData)
    }

    then: "user is created"
    response.statusCode == 201
    response.json.id == 456

    and: "request contains auth header"
    testDsl.verify("/users") {
        wasCalled(1)
        withMethod("POST")
        withHeader("Authorization", "Bearer token123")
        withHeader("Content-Type")
        withJsonBody { json ->
            json.name == "Bob" &&
            json.email == "bob@example.com" &&
            json.roles.contains("user")
        }
    }
}
```

Chapter 90. JUnit 5 Integration

```
import org.junit.jupiter.api.*

class UserApiTest {

    private HttpClientTestDsl testDsl
    private GroovyHttpClient client

    @BeforeEach
    void setup() {
        testDsl = HttpClientTestDsl.create()
        testDsl.start()
        client = testDsl.createClient()
    }

    @AfterEach
    void teardown() {
        testDsl.stop()
    }

    @Test
    void shouldFetchUser() {
        // given
        testDsl.when("/users/123").respondWith {
            status 200
            json([id: 123, name: "Alice"])
        }

        // when
        def response = client.getSync("/users/123")

        // then
        Assertions.assertEquals(200, response.statusCode)
        Assertions.assertEquals("Alice", response.json.name)

        // verify
        testDsl.verify("/users/123") {
            wasCalled(1)
            withMethod("GET")
        }
    }

    @Test
    void shouldHandleNotFound() {
        // given
        testDsl.when("/users/999").respondWith {
            status 404
            json([error: "User not found"])
```

```
        }

        // when
        def thrown = Assertions.assertThrows(
            GroovyHttpClient.HttpResponseException.class,
            () -> client.getSync("/users/999")
        )

        // then
        Assertions.assertEquals(404, thrown.statusCode)
    }
}
```

Chapter 91. Advanced Patterns

91.1. Testing Async Operations

```
def "should handle async requests"() {
    given:
    testDsl.when("/data").respondWithJson([value: 42])

    when: "making async request"
    def future = client.get("/data")

    and: "waiting for completion"
    def response = future.get()

    then:
    response.statusCode == 200
    response.json.value == 42

    and:
    testDsl.verify("/data") {
        wasCalled(1)
    }
}
```

=# Multiple Calls to Same Endpoint

```
def "should track multiple calls"() {
    given:
    testDsl.when("/counter").respondWith {
        status 200
        json([count: 1])
    }

    when: "making multiple requests"
    3.times { client.getSync("/counter") }

    then: "all calls are recorded"
    testDsl.verify("/counter") {
        wasCalled(3)
    }

    and: "request history is available"
    def requests = testDsl.getRequestsFor("/counter")
    requests.size() == 3
    requests.every { it.method == "GET" }
}
```

91.2. Testing Request Order

```
def "should verify request order"() {
    given:
    testDsl.when("/step1").respondWithStatus(200)
    testDsl.when("/step2").respondWithStatus(200)
    testDsl.when("/step3").respondWithStatus(200)

    when:
    client.getSync("/step1")
    client.getSync("/step2")
    client.getSync("/step3")

    then:
    def allRequests = testDsl.recordedRequests
    allRequests[0].path == "/step1"
    allRequests[1].path == "/step2"
    allRequests[2].path == "/step3"
}
```

91.3. Testing Error Recovery

```
def "should retry on server error"() {
    given: "a server that returns 503"
    testDsl.when("/flaky").respondWith {
        status 503
        json([error: "Service unavailable"])
    }

    and: "a client with retry policy"
    def retryClient = testDsl.createClient {
        retryPolicy = new RetryPolicy.Config().tap {
            maxAttempts(3)
            on5xx(true)
        }.build()
    }

    when: "making a request"
    try {
        retryClient.getSync("/flaky")
    } catch (ignored) {}

    then: "multiple attempts were made"
    def requests = testDsl.getRequestsFor("/flaky")
    requests.size() >= 1  // At least one attempt
}
```

91.4. Testing Multipart Uploads

```
def "should upload file via multipart"() {
    given: "a file upload endpoint"
    testDsl.when("/upload").respondWith {
        status 201
        json([uploaded: true, filename: "document.pdf"])
    }

    when: "uploading a file"
    def response = client.postMultipartSync("/upload") {
        part {
            name "file"
            filename "document.pdf"
            contentType "application/pdf"
            data pdfBytes
        }
        part {
            name "description"
            content "Important document"
        }
    }

    then: "upload succeeds"
    response.statusCode == 201
    response.json.uploaded == true

    and: "request was received"
    testDsl.verify("/upload") {
        wasCalled(1)
        withMethod("POST")
        withHeader("Content-Type")  // multipart/form-data
    }
}
```

91.5. Testing Circuit Breaker Behavior

```
def "should open circuit breaker on failures"() {
    given: "a failing endpoint"
    testDsl.when("/failing").respondWith {
        status 503
        text "Service Unavailable"
    }

    and: "a client with low circuit breaker threshold"
    def fragileClient = testDsl.createClient {
        failureThreshold = 3
        resetTimeoutMs = 60_000
```

```
    }

    when: "making requests until circuit opens"
    3.times {
        try { fragileClient.getSync("/failing") } catch (ignored) {}
    }

    then: "circuit breaker opens"
    def thrown = null
    try {
        fragileClient.getSync("/failing")
    } catch (GroovyHttpClient.CircuitOpenException e) {
        thrown = e
    }
    thrown != null

    and: "requests were attempted"
    testDsl.getRequestsFor("/failing").size() == 3
}
```

Chapter 92. Best Practices

=# Use Descriptive Path Names

```
// ✅ Good: Descriptive, intention-revealing
testDsl.when("/users/123").respondWithJson([id: 123, name: "Alice"])
testDsl.when("/orders/search").respondWithJson([results: []])

// ❌ Avoid: Generic, unclear
testDsl.when("/endpoint1").respondWithJson([data: "x"])
testDsl.when("/test").respondWithJson([result: true])
```

92.1. Reset State Between Tests

```
def cleanup() {
    testDsl.reset()  // Clears all endpoints and request history
}
```

92.2. Use Realistic Response Data

```
// ✅ Realistic
testDsl.when("/users/123").respondWith {
    json([
        id: 123,
        name: "Alice Smith",
        email: "alice@example.com",
        created_at: "2024-01-15T10:30:00Z",
        roles: ["user", "admin"],
        metadata: [
            last_login: "2025-01-24T15:45:00Z",
            login_count: 42
        ]
    ])
}

// ❌ Minimal (harder to catch bugs)
testDsl.when("/users/123").respondWithJson([id: 1, name: "A"])
```

=# Verify Request Content

```
// ✅ Verify important aspects
testDsl.verify("/users") {
    wasCalled(1)
    withMethod("POST")
```

```
    withHeader("Authorization")
    withJsonBody { it.name && it.email }
}

// □ Only verify call count
testDsl.verify("/users") {
    wasCalled(1)
}
```

=# Use Helper Methods

Extract common endpoint definitions:

```
class ApiTestSupport {
    static void mockUserEndpoint(HttpClientTestDsl dsl, Map userData) {
        dsl.when("/users/${userData.id}").respondWith {
            status 200
            json(userData)
        }
    }

    static void mockAuthenticatedEndpoint(HttpClientTestDsl dsl, String path) {
        dsl.when(path).respondWith {
            status 401
            json([error: "Unauthorized"])
        }
    }
}

// Usage
mockUserEndpoint(testDsl, [id: 123, name: "Alice"])
```

=# Test Both Success and Failure Cases

```
def "should handle success"() {
    given:
    testDsl.when("/data").respondWithJson([value: 42])

    expect:
    client.getSync("/data").json.value == 42
}

def "should handle 404"() {
    given:
    testDsl.when("/missing").respondWith {
        status 404
        json([error: "Not found"])
    }
```

```
    when:
    client.getSync("/missing")

    then:
    thrown(GroovyHttpClient.HttpResponseException)
}

def "should handle timeout"() {
    // Test timeout scenarios
}

def "should handle malformed JSON"() {
    // Test parse error scenarios
}
```

Chapter 93. Limitations

HttpClientTestDsl is designed for straightforward request/response testing:

Not Supported	WebSocket connections
Not Supported	Server-sent events (SSE)
Not Supported	HTTP/2 server push
Not Supported	Chunked transfer encoding
Limited	Response varies by request content (use conditionals in DSL)

For these scenarios, consider integration testing with real services or specialized tools.

Chapter 94. Troubleshooting

=# Port Already in Use

```
// Solution: Let server choose port automatically (default)
def testDsl = HttpClientTestDsl.create()  // Port 0 = auto-assign
testDsl.start()
println "Server on port: ${testDsl.port}"
```

=# Verification Fails

```
// Problem: Path mismatch
testDsl.verify("/api/users")  // Fails

// Solution: Check actual recorded paths
def requests = testDsl.recordedRequests
requests.each { println it.path }  // Shows "/users", not "/api/users"
```

=# Headers Not Matching

```
// Headers are case-insensitive
testDsl.verify("/data") {
    withHeader("content-type")      // ✓ Works
    withHeader("Content-Type")      // ✓ Works
    withHeader("CONTENT-TYPE")      // ✓ Works
}
```

=# JSON Body Not Matching

```
// Problem: Exact string comparison
testDsl.verify("/data") {
    withBody('{"name":"Alice"}')  // May fail due to whitespace/order
}

// Solution: Use JSON matcher
testDsl.verify("/data") {
    withJsonBody { json ->
        json.name == "Alice"
    }
}
```

=# Request History Growing

```
// Problem: History accumulates across tests
```

```
// Solution: Reset between tests
def cleanup() {
    testDsl.reset()
}

// Or: Check specific requests
def userRequests = testDsl.getRequestsFor("/users")
```

Chapter 95. Performance Tips

Shared Server	Reuse server across tests (see Spock example)
Minimal Endpoints	Define only needed endpoints per test
Reset History	Clear history between tests to avoid memory growth
Auto Cleanup	Use try-with-resources or cleanup methods

Chapter 96. Next Steps

Explore related testing tools:

- MockHttpServer — Validation-focused testing
- HttpClientTestTrait — Spock trait integration
- GroovyHttpClient — Full client API reference
- Real-World Examples — Complete test suites

HttpClientTestTrait

Chapter 97. Overview

`HttpClientTestTrait` is a Groovy trait that seamlessly integrates `HttpClientTestDsl` into Spock specifications. It eliminates boilerplate setup code, provides convenient helper methods, and manages the test server lifecycle automatically.

> Mix in testing superpowers. One trait, zero boilerplate.

97.1. Key Benefits

Zero Boilerplate	No manual server creation or lifecycle management
Auto Cleanup	Server automatically stopped after tests
Convenient Helpers	Shorthand methods for common operations
Shared Server	Single server instance for all tests (performance)
Familiar API	Exposes `HttpClientTestDsl` methods directly
Type Safe	Full IDE autocomplete and compile-time checking

Chapter 98. Quick Start

98.1. Basic Usage

```
import org.softwood.http.HttpClientTestTrait
import spock.lang.Specification

class UserApiSpec extends Specification implements HttpClientTestTrait {

    def "should fetch user by ID"() {
        given: "a mocked user endpoint"
        mockEndpoint("/users/123") {
            status 200
            json([id: 123, name: "Alice"])
        }

        when: "fetching user"
        def client = createTestClient()
        def response = client.getSync("/users/123")

        then: "user data is returned"
        response.statusCode == 200
        response.json.name == "Alice"

        and: "request was made"
        verifyRequest("/users/123").wasCalled()
    }
}
```

That's it! No setup/cleanup, no manual server management.

98.2. What the Trait Provides

The trait automatically:

- Creates `HttpClientTestDsl` instance
- Starts the test server
- Manages server lifecycle
- Provides helper methods
- Cleans up after tests

Chapter 99. Trait Methods

99.1. Server Access

Get Test Server:

```
// Access underlying HttpClientTestDsl
def testServer = this.testServer
println "Server running on: ${testServer.baseUrl}"
```

Get Base URL:

```
def url = testServerUrl
// e.g., "http://localhost:54321"
```

99.2. Client Creation

Create Test Client:

```
// Default testing config
def client = createTestClient()

// With custom config
def client = createTestClient {
    requestTimeout = Duration.ofSeconds(60)
    enableLogging = true
    failureThreshold = 3
}
```

99.3. Endpoint Mocking

Simple Endpoint:

```
// Returns MockEndpoint for chaining
def endpoint = mockEndpoint("/api/data")

endpoint.respondWith {
    status 200
    json([data: "value"])
}
```

With Response Configuration:

```
mockEndpoint("/users") {
    status 200
    json([
        users: [[id: 1], [id: 2]]
    ])
}
```

This is equivalent to:

```
testServer.when("/users").respondWith {
    status 200
    json([users: [[id: 1], [id: 2]]])
}
```

99.4. Request Verification

Verify Requests:

```
verifyRequest("/api/data")
    .wasCalled()
    .withMethod("GET")
    .withHeader("Accept", "application/json")
```

Get Recorded Requests:

```
// All requests
def allRequests = recordedRequests

// Requests for specific path
def userRequests = getRecordedRequests("/users")
```

99.5. Server Management

Reset Server State:

```
// Clear endpoints and request history
resetTestServer()
```

Manual Server Cleanup (optional):

```
// Called automatically in cleanupSpec
// Only call manually if needed
```

```
stopTestServer()
```

Chapter 100. Complete Example

```
import org.softwood.http.HttpClientTestTrait
import org.softwood.http.GroovyHttpClient
import spock.lang.Specification

class UserServiceSpec extends Specification implements HttpClientTestTrait {

    UserService userService

    def setup() {
        def client = createTestClient()
        userService = new UserService(client)
    }

    def cleanup() {
        resetTestServer()
    }

    def "should list all users"() {
        given: "users endpoint returns list"
        mockEndpoint("/users") {
            status 200
            json([
                users: [
                    [id: 1, name: "Alice", email: "alice@example.com"],
                    [id: 2, name: "Bob", email: "bob@example.com"]
                ],
                total: 2
            ])
        }

        when: "fetching users"
        def users = userService.listUsers()

        then: "users are returned"
        users.size() == 2
        users[0].name == "Alice"
        users[1].name == "Bob"

        and: "correct request was made"
        verifyRequest("/users")
            .wasCalled()
            .withMethod("GET")
    }

    def "should get user by ID"() {
        given: "user endpoint returns data"
        mockEndpoint("/users/123") {
```

```
            status 200
            json([
                id: 123,
                name: "Alice",
                email: "alice@example.com"
            ])
        }

        when: "fetching user"
        def user = userService.getUser(123)

        then: "user data is correct"
        user.id == 123
        user.name == "Alice"
        user.email == "alice@example.com"
    }

    def "should create new user"() {
        given: "create endpoint returns created user"
        mockEndpoint("/users") {
            status 201
            json([
                id: 456,
                name: "Charlie",
                email: "charlie@example.com"
            ])
            header "Location", "/users/456"
        }

        when: "creating user"
        def user = userService.createUser("Charlie", "charlie@example.com")

        then: "user is created"
        user.id == 456
        user.name == "Charlie"

        and: "correct request was made"
        verifyRequest("/users")
            .wasCalled()
            .withMethod("POST")
            .withHeader("Content-Type", "application/json")
            .withBodyContaining("Charlie")
    }

    def "should handle user not found"() {
        given: "endpoint returns 404"
        mockEndpoint("/users/999") {
            status 404
            json([error: "User not found"])
        }
```

```
        when: "fetching non-existent user"
        userService.getUser(999)

        then: "throws exception"
        thrown(UserNotFoundException)
    }

    def "should handle server errors"() {
        given: "endpoint returns 500"
        mockEndpoint("/users") {
            status 500
            json([error: "Internal server error"])
        }

        when: "fetching users"
        userService.listUsers()

        then: "throws exception"
        thrown(ServerException)

        and: "request was attempted"
        verifyRequest("/users").wasCalled()
    }
}

// Example service class
class UserService {
    private final GroovyHttpClient client

    UserService(GroovyHttpClient client) {
        this.client = client
    }

    List<User> listUsers() {
        def response = client.getSync("/users")
        response.json.users.collect { new User(it) }
    }

    User getUser(int id) {
        try {
            def response = client.getSync("/users/${id}")
            new User(response.json)
        } catch (HttpResponseException e) {
            if (e.statusCode == 404) {
                throw new UserNotFoundException("User ${id} not found")
            }
            throw e
        }
    }

    User createUser(String name, String email) {
```

```
        def response = client.postSync("/users", null) {
            json([name: name, email: email])
        }
        new User(response.json)
    }
}

class User {
    int id
    String name
    String email

    User(Map data) {
        this.id = data.id as int
        this.name = data.name
        this.email = data.email
    }
}

class UserNotFoundException extends RuntimeException {
    UserNotFoundException(String message) {
        super(message)
    }
}

class ServerException extends RuntimeException {
    ServerException(String message) {
        super(message)
    }
}
```

Chapter 101. Advanced Usage

101.1. Multiple Endpoints

```
def "should fetch user and their posts"() {
    given: "user endpoint"
    mockEndpoint("/users/1") {
        json([id: 1, name: "Alice"])
    }

    and: "posts endpoint"
    mockEndpoint("/users/1/posts") {
        json([
            posts: [
                [id: 1, title: "First Post"],
                [id: 2, title: "Second Post"]
            ]
        ])
    }

    when: "fetching user and posts"
    def client = createTestClient()
    def user = client.getSync("/users/1").json
    def posts = client.getSync("/users/1/posts").json.posts

    then:
    user.name == "Alice"
    posts.size() == 2

    and: "both requests were made"
    verifyRequest("/users/1").wasCalled()
    verifyRequest("/users/1/posts").wasCalled()
}
```

101.2. Error Scenarios

```
def "should retry on server errors"() {
    given: "endpoint that returns 500"
    mockEndpoint("/flaky") {
        status 500
        body "Internal error"
    }

    and: "client with retry policy"
    def client = createTestClient {
        retryPolicy = new RetryPolicy.Config().tap {
            maxAttempts(3)
```

```
            backoffExponential(100, 1000, false)
            on5xx(true)
        }.build()
    }

    when: "making request"
    client.getSync("/flaky")

    then: "throws exception after retries"
    thrown(HttpResponseException)

    and: "multiple attempts were made"
    def requests = getRecordedRequests("/flaky")
    requests.size() >= 1
}
```

=# Testing with Headers

```
def "should send authentication header"() {
    given: "secured endpoint"
    mockEndpoint("/admin/users") {
        status 200
        json([users: []])
    }

    and: "client with auth"
    def client = createTestClient()
    client.setHeader("Authorization", "Bearer secret123")

    when: "accessing secured endpoint"
    def response = client.getSync("/admin/users")

    then:
    response.statusCode == 200

    and: "auth header was sent"
    verifyRequest("/admin/users")
        .wasCalled()
        .withHeader("Authorization", "Bearer secret123")
}
```

101.3. Testing with Cookies

```
def "should maintain session cookie"() {
    given: "login endpoint that sets cookie"
    mockEndpoint("/login") {
        status 200
        json([success: true])
```

```
    }

    and: "profile endpoint"
    mockEndpoint("/profile") {
        status 200
        json([name: "Alice"])
    }

    when: "logging in"
    def client = createTestClient()
    client.postSync("/login", null) {
        form(username: "alice", password: "secret")
    }

    and: "setting session cookie"
    client.addCookie("session", "abc123")

    and: "accessing profile"
    client.getSync("/profile")

    then: "requests were made"
    verifyRequest("/login").wasCalled()
    verifyRequest("/profile").wasCalled()
}
```

Chapter 102. Data-Driven Tests

```
def "should handle different status codes"() {
    given: "endpoint returning status"
    mockEndpoint("/api") {
        status statusCode
        body message
    }

    when: "making request"
    def client = createTestClient()
    try {
        client.getSync("/api")
        success = true
    } catch (HttpResponseException e) {
        success = false
    }

    then:
    success == shouldSucceed

    where:
    statusCode | message        | shouldSucceed
    200        | "OK"           | true
    201        | "Created"      | true
    204        | ""             | true
    400        | "Bad Request"  | false
    404        | "Not Found"    | false
    500        | "Server Error" | false
}
```

```
def "should handle different content types"() {
    given: "endpoint with content type"
    mockEndpoint("/data") {
        status 200
        contentType contentType
        body responseBody
    }

    when: "fetching data"
    def client = createTestClient()
    def response = client.getSync("/data")

    then:
    response.contentType == contentType

    where:
    contentType             | responseBody
```

```
    "application/json"      | '{"key":"value"}'
    "text/plain"            | "Hello, World!"
    "text/html"             | "<h1>Title</h1>"
    "application/xml"       | "<root><item/></root>"
}
```

Chapter 103. Testing Patterns

103.1. Testing Paginated APIs

```
def "should fetch all pages"() {
    given: "paginated endpoints"
    mockEndpoint("/users?page=1") {
        json([
            users: [[id: 1], [id: 2]],
            page: 1,
            total_pages: 2
        ])
    }

    mockEndpoint("/users?page=2") {
        json([
            users: [[id: 3], [id: 4]],
            page: 2,
            total_pages: 2
        ])
    }

    when: "fetching all pages"
    def client = createTestClient()
    def allUsers = []

    (1..2).each { page ->
        def response = client.getSync("/users") {
            query(page: page)
        }
        allUsers.addAll(response.json.users)
    }

    then:
    allUsers.size() == 4

    and: "both pages were fetched"
    verifyRequest("/users").wasCalled(2)
}
```

103.2. Testing Error Recovery

```
def "should fall back on error"() {
    given: "primary endpoint fails"
    mockEndpoint("/primary") {
        status 503
        body "Service unavailable"
```

```
    }

    and: "fallback endpoint succeeds"
    mockEndpoint("/fallback") {
        status 200
        json([data: "fallback"])
    }

    when: "service tries primary then fallback"
    def result = service.fetchWithFallback()

    then:
    result.data == "fallback"

    and: "both were attempted"
    verifyRequest("/primary").wasCalled()
    verifyRequest("/fallback").wasCalled()
}
```

103.3. Testing Rate Limiting

```
def "should respect rate limits"() {
    given: "rate-limited endpoint"
    mockEndpoint("/api/data") {
        status 429
        json([error: "Rate limit exceeded"])
        header "X-RateLimit-Reset", "60"
    }

    when: "making request"
    def client = createTestClient()
    client.getSync("/api/data")

    then: "throws exception"
    def ex = thrown(HttpResponseException)
    ex.statusCode == 429

    and: "request was made"
    verifyRequest("/api/data").wasCalled()
}
```

Chapter 104. Trait Implementation Details

The trait provides:

```
trait HttpClientTestTrait {
    @Shared
    private HttpClientTestDsl _testServer

    // Lazy initialization
    HttpClientTestDsl getTestServer() {
        if (_testServer == null) {
            _testServer = HttpClientTestDsl.create()
            _testServer.start()
        }
        _testServer
    }

    // Convenience methods
    String getTestServerUrl() { testServer.baseUrl }

    GroovyHttpClient createTestClient(Closure config = null) {
        testServer.createClient(config)
    }

    MockEndpoint mockEndpoint(String path) {
        testServer.when(path)
    }

    MockEndpoint mockEndpoint(String path, Closure config) {
        testServer.when(path).respondWith(config)
    }

    RequestVerification verifyRequest(String path) {
        testServer.verify(path)
    }

    void resetTestServer() {
        testServer.reset()
    }

    List<RecordedRequest> getRecordedRequests(String path = null) {
        path ? testServer.getRequestsFor(path) : testServer.recordedRequests
    }

    void stopTestServer() {
        _testServer?.stop()
        _testServer = null
    }
}
```

Chapter 105. Comparison: Direct vs Trait

Without Trait:

```
class ApiSpec extends Specification {
    HttpClientTestDsl testDsl
    GroovyHttpClient client

    def setup() {
        testDsl = HttpClientTestDsl.create()
        testDsl.start()
        client = testDsl.createClient()
    }

    def cleanup() {
        testDsl.stop()
    }

    def "test"() {
        testDsl.when("/api").respondWithStatus(200)
        def response = client.getSync("/api")
        testDsl.verify("/api").wasCalled()
    }
}
```

With Trait:

```
class ApiSpec extends Specification implements HttpClientTestTrait {

    def "test"() {
        mockEndpoint("/api") { status 200 }
        def response = createTestClient().getSync("/api")
        verifyRequest("/api").wasCalled()
    }
}
```

Much cleaner!

Chapter 106. Best Practices

106.1. Reset Between Tests

```
def cleanup() {
    resetTestServer()
}
```

This ensures tests don't interfere with each other.

=# Group Related Tests

```
class UserApiSpec extends Specification implements HttpClientTestTrait {

    // User CRUD tests
    def "should create user"() { /* ... */ }
    def "should read user"() { /* ... */ }
    def "should update user"() { /* ... */ }
    def "should delete user"() { /* ... */ }

    // Error handling tests
    def "should handle 404"() { /* ... */ }
    def "should handle 500"() { /* ... */ }

    // Authentication tests
    def "should require auth"() { /* ... */ }
    def "should reject bad token"() { /* ... */ }
}
```

106.2. Use Helper Methods

```
class ApiSpec extends Specification implements HttpClientTestTrait {

    private mockSuccessfulUser() {
        mockEndpoint("/users/123") {
            json([id: 123, name: "Alice"])
        }
    }

    private mockMissingUser() {
        mockEndpoint("/users/999") {
            status 404
            json([error: "Not found"])
        }
    }
```

```
    def "test 1"() {
        given:
        mockSuccessfulUser()
        // ...
    }

    def "test 2"() {
        given:
        mockMissingUser()
        // ...
    }
}
```

106.3. Document Test Intent

```
def "should create user with all required fields"() {
    given: "a valid user payload"
    mockEndpoint("/users") {
        status 201
        json([id: 123, name: "Alice", email: "alice@example.com"])
    }

    when: "creating user with name and email"
    def client = createTestClient()
    def response = client.postSync("/users", null) {
        json([
            name: "Alice",
            email: "alice@example.com"
        ])
    }

    then: "user is created successfully"
    response.statusCode == 201
    response.json.id == 123

    and: "all required fields are present"
    response.json.name == "Alice"
    response.json.email == "alice@example.com"

    and: "correct API call was made"
    verifyRequest("/users")
        .wasCalled()
        .withMethod("POST")
        .withHeader("Content-Type", "application/json")
        .withBodyContaining("Alice")
        .withBodyContaining("alice@example.com")
}
```

Chapter 107. Limitations

The trait uses a **shared** test server instance:

- Faster (no server restart between tests)
- Must call `resetTestServer()` to avoid test interference
- Port is assigned once and reused

For tests requiring separate servers, use `HttpClientTestDsl` directly.

Chapter 108. Troubleshooting

108.1. Tests Interfering with Each Other

Problem: Test B fails because Test A's mocked endpoints still exist.

Solution: Call `resetTestServer()` in `cleanup()`:

```
def cleanup() {
    resetTestServer()
}
```

108.2. "Server not started" Error

Problem: Trying to use `testServerUrl` before server is initialized.

Solution: Access `testServer` property first to trigger initialization:

```
def "test"() {
    given:
    testServer  // Trigger initialization
    def url = testServerUrl  // Now available
}
```

Or just use the helper methods which handle this automatically.

108.3. Port Conflicts

The trait uses a single shared server for all tests, so port conflicts shouldn't occur. If you need multiple servers, use `HttpClientTestDsl` directly.

Chapter 109. Next Steps

You now have all the tools for comprehensive HTTP client testing:

- Real-World Examples — Complete, working test suites
- Testing Best Practices — Patterns and anti-patterns
- Architecture Overview — Understanding the full picture

Real-World Examples

Chapter 110. Overview

This chapter provides practical, real-world examples demonstrating common usage patterns for GroovyHttpClient. Each example includes complete, runnable code that you can adapt for your own projects.



All examples in this chapter are adapted from actual tests in the project. You can find more examples in the test suite at:

- `src/test/groovy/org/softwood/http/clientTests/` - Client functionality tests
- `src/test/groovy/org/softwood/http/testingFrameworkTests/` - Testing framework examples

These tests serve as both documentation and validation of the library's features.

Chapter 111. Basic HTTP Operations

111.1. Simple GET Request

```
import org.softwood.http.GroovyHttpClient

// Create client
def client = new GroovyHttpClient("https://api.github.com")

// Make GET request
def response = client.getSync("/users/octocat")

// Access response data
println "Status: ${response.statusCode}"
println "User: ${response.json.name}"
println "Location: ${response.json.location}"

// Response: 200
// User: The Octocat
// Location: San Francisco
```

111.2. POST with JSON Body

```
import org.softwood.http.GroovyHttpClient

def client = new GroovyHttpClient("https://api.example.com")

// Create user
def userData = [
    name: "Alice Smith",
    email: "alice@example.com",
    roles: ["user", "admin"],
    metadata: [
        department: "Engineering",
        start_date: "2025-01-24"
    ]
]

def response = client.postSync("/users", null) {
    json(userData)
}

println "Created user ID: ${response.json.id}"
println "Status: ${response.json.status}"
```

111.3. PUT Request to Update Resource

```
def client = new GroovyHttpClient("https://api.example.com")

// Update user
def updates = [
    email: "alice.smith@example.com",
    roles: ["user", "admin", "moderator"]
]

def response = client.putSync("/users/123", null) {
    json(updates)
    header "Authorization", "Bearer ${authToken}"
}

assert response.statusCode == 200
println "User updated: ${response.json}"
```

111.4. DELETE Request

```
def client = new GroovyHttpClient("https://api.example.com")

def response = client.deleteSync("/users/123") {
    header "Authorization", "Bearer ${authToken}"
}

assert response.statusCode == 204  // No Content
println "User deleted successfully"
```

Chapter 112. Authentication Patterns

112.1. Bearer Token Authentication

```
import org.softwood.http.GroovyHttpClient

def client = new GroovyHttpClient("https://api.example.com")

// Set default auth header for all requests
def authToken = "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
client.setHeader("Authorization", "Bearer ${authToken}")

// All requests now include auth header
def users = client.getSync("/users").json
def posts = client.getSync("/posts").json
def profile = client.getSync("/profile").json
```

112.2. API Key Authentication

```
def client = new GroovyHttpClient("https://api.example.com")

// API key in header
client.setHeader("X-API-Key", "sk_live_abc123xyz789")

// Or in query parameter
def response = client.getSync("/data") {
    query(api_key: "sk_live_abc123xyz789")
}
```

112.3. Basic Authentication

```
def client = new GroovyHttpClient("https://api.example.com")

def username = "alice"
def password = "secret123"
def credentials = "${username}:${password}".bytes.encodeBase64().toString()

client.setHeader("Authorization", "Basic ${credentials}")

def response = client.getSync("/protected/resource")
```

112.4. OAuth 2.0 Flow

```
class OAuth2Client {
    private final GroovyHttpClient client
    private String accessToken
    private long tokenExpiry

    OAuth2Client(String baseUrl, String clientId, String clientSecret) {
        this.client = new GroovyHttpClient(baseUrl)
        refreshToken(clientId, clientSecret)
    }

    private void refreshToken(String clientId, String clientSecret) {
        def tokenResponse = client.postSync("/oauth/token", null) {
            form(
                grant_type: "client_credentials",
                client_id: clientId,
                client_secret: clientSecret
            )
        }

        this.accessToken = tokenResponse.json.access_token
        this.tokenExpiry = System.currentTimeMillis() +
                           (tokenResponse.json.expires_in * 1000)
    }

    def getProtectedResource(String path) {
        // Auto-refresh if token expired
        if (System.currentTimeMillis() >= tokenExpiry) {
            refreshToken(clientId, clientSecret)
        }

        return client.getSync(path) {
            header "Authorization", "Bearer ${accessToken}"
        }
    }
}

// Usage
def oauth = new OAuth2Client(
    "https://api.example.com",
    "client_id_123",
    "client_secret_xyz"
)

def data = oauth.getProtectedResource("/api/data").json
```

Chapter 113. Query Parameters

113.1. Simple Query Parameters

```
def client = new GroovyHttpClient("https://api.example.com")

// Search with filters
def response = client.getSync("/products") {
    query(
        category: "electronics",
        min_price: 100,
        max_price: 500,
        in_stock: true,
        sort: "price_asc"
    )
}

// URL:
/products?category=electronics&min_price=100&max_price=500&in_stock=true&sort=price_as
c

def products = response.json.results
products.each { product ->
    println "${product.name}: $${product.price}"
}
```

113.2. Array Query Parameters

```
// Multiple values for same parameter
def response = client.getSync("/articles") {
    query(
        tags: ["groovy", "http", "api"],
        status: ["published", "featured"]
    )
}

// URL: /articles?tags=groovy&tags=http&tags=api&status=published&status=featured
```

113.3. Pagination

```
def client = new GroovyHttpClient("https://api.example.com")

def fetchAllPages(String endpoint) {
    def allResults = []
    def page = 1
```

```
    def hasMore = true

    while (hasMore) {
        def response = client.getSync(endpoint) {
            query(page: page, per_page: 100)
        }

        def data = response.json
        allResults.addAll(data.results)

        hasMore = data.has_more
        page++
    }

    return allResults
}

// Fetch all users across multiple pages
def allUsers = fetchAllPages("/users")
println "Total users: ${allUsers.size()}"
```

Chapter 114. File Operations

114.1. Upload Single File

```
import org.softwood.http.GroovyHttpClient

def client = new GroovyHttpClient("https://api.example.com")

// Read file
def file = new File("report.pdf")
def fileBytes = file.bytes

// Upload using DSL
def response = client.postMultipartSync("/documents") {
    part {
        name "file"
        filename file.name
        contentType "application/pdf"
        data fileBytes
    }

    part {
        name "title"
        content "Monthly Report"
    }

    part {
        name "category"
        content "reports"
    }

    header "Authorization", "Bearer ${authToken}"
}

println "Document uploaded: ${response.json.document_id}"
```

114.2. Upload Multiple Files

```
import org.softwood.http.MultipartPart

def client = new GroovyHttpClient("https://api.example.com")

// Prepare files
def files = [
    new File("invoice.pdf"),
    new File("receipt.pdf"),
    new File("contract.pdf")
```

```
]

// Build parts
def parts = files.collect { file ->
    MultipartPart.bytes(
        "documents",  // Same name for all
        file.name,
        "application/pdf",
        file.bytes
    )
}

// Add metadata part
parts << MultipartPart.json(
    "metadata",
    groovy.json.JsonOutput.toJson([
        batch_id: UUID.randomUUID().toString(),
        uploaded_by: "alice",
        timestamp: new Date().format("yyyy-MM-dd'T'HH:mm:ss'Z'")
    ])
)

// Upload
def response = client.postMultipartSync("/batch-upload", parts) {
    header "Authorization", "Bearer ${authToken}"
}

println "Uploaded ${response.json.count} files"
```

114.3. Download File

```
def client = new GroovyHttpClient("https://api.example.com")

// Download file
def fileBytes = client.downloadBytesSync("/files/document.pdf") {
    header "Authorization", "Bearer ${authToken}"
}

// Save to disk
new File("downloaded_document.pdf").bytes = fileBytes

println "Downloaded ${fileBytes.length} bytes"
```

114.4. Download with Progress Tracking

```
import java.util.concurrent.atomic.AtomicLong
```

```
def client = new GroovyHttpClient("https://api.example.com")

// Add progress tracking interceptor
def bytesDownloaded = new AtomicLong(0)

client.addInterceptor { ctx, next ->
    def start = System.currentTimeMillis()

    next.call(ctx).thenApply { response ->
        def elapsed = System.currentTimeMillis() - start
        def size = response.bytes.length
        bytesDownloaded.addAndGet(size)

        println "Downloaded ${size} bytes in ${elapsed}ms"
        response
    }
}

// Download file
def fileBytes = client.downloadBytesSync("/large-file.zip")
println "Total downloaded: ${bytesDownloaded.get()} bytes"
```

Chapter 115. Error Handling

115.1. Handling HTTP Errors

```
import org.softwood.http.GroovyHttpClient

def client = new GroovyHttpClient("https://api.example.com")

try {
    def response = client.getSync("/users/999")
    println "User: ${response.json}"

} catch (GroovyHttpClient.HttpResponseException e) {
    switch (e.statusCode) {
        case 400:
            println "Bad request: ${e.message}"
            break
        case 401:
            println "Unauthorized - check your credentials"
            break
        case 404:
            println "User not found"
            break
        case 500:
            println "Server error - try again later"
            break
        default:
            println "HTTP error ${e.statusCode}: ${e.message}"
    }
} catch (IOException e) {
    println "Network error: ${e.message}"
} catch (java.net.http.HttpTimeoutException e) {
    println "Request timed out"
}
```

115.2. Retry with Exponential Backoff

```
import org.softwood.http.GroovyHttpClient.RetryPolicy
import org.softwood.http.SecurityConfig

def retryPolicy = new RetryPolicy.Config().tap {
    maxAttempts(5)
    backoffExponential(200, 10_000, true)
    onTimeouts(true)
    on5xx(true)
    onIoExceptions(true)
}.build()
```

```
def config = SecurityConfig.production("https://api.example.com")
    .withOverrides([retryPolicy: retryPolicy])

def client = new GroovyHttpClient(config)

// This will retry up to 5 times on failures
def response = client.getSync("/flaky-endpoint")
```

115.3. Circuit Breaker Pattern

```
import org.softwood.http.SecurityConfig
import org.softwood.http.GroovyHttpClient

// Configure circuit breaker
def config = SecurityConfig.production("https://api.example.com")
    .withOverrides([
        failureThreshold: 5,      // Open after 5 failures
        resetTimeoutMs: 30_000    // Try recovery after 30 seconds
    ])

def client = new GroovyHttpClient(config)

def fetchData() {
    try {
        return client.getSync("/data").json
    } catch (GroovyHttpClient.CircuitOpenException e) {
        println "Service unavailable - using cached data"
        return cacheService.get("data")
    } catch (Exception e) {
        println "Request failed: ${e.message}"
        return null
    }
}

// Multiple calls - circuit breaker protects against cascading failures
10.times {
    def data = fetchData()
    println "Data: ${data}"
    Thread.sleep(1000)
}
```

115.4. Graceful Degradation

```
class ApiClient {
    private final GroovyHttpClient primaryClient
    private final GroovyHttpClient backupClient
```

```
    private final CacheService cache

    ApiClient(String primaryUrl, String backupUrl) {
        this.primaryClient = new GroovyHttpClient(primaryUrl)
        this.backupClient = new GroovyHttpClient(backupUrl)
        this.cache = new CacheService()
    }

    def fetchUserData(String userId) {
        // Try primary service
        try {
            def data = primaryClient.getSync("/users/${userId}").json
            cache.put("user:${userId}", data)
            return data
        } catch (GroovyHttpClient.CircuitOpenException e) {
            println "Primary circuit open - trying backup"
        } catch (Exception e) {
            println "Primary failed: ${e.message}"
        }

        // Try backup service
        try {
            def data = backupClient.getSync("/users/${userId}").json
            cache.put("user:${userId}", data)
            return data
        } catch (Exception e) {
            println "Backup failed: ${e.message}"
        }

        // Fall back to cache
        def cached = cache.get("user:${userId}")
        if (cached) {
            println "Returning cached data"
            return cached
        }

        throw new RuntimeException("All data sources unavailable")
    }
}
```

Chapter 116. Async Operations

116.1. Concurrent Requests

```
import org.softwood.http.GroovyHttpClient
import java.util.concurrent.CompletableFuture

def client = new GroovyHttpClient("https://api.example.com")

// Launch multiple async requests
def futures = (1..10).collect { id ->
    client.get("/users/${id}").thenApply { response ->
        response.json
    }
}

// Wait for all to complete
def allUsers = CompletableFuture.allOf(*futures).thenApply {
    futures.collect { it.join() }
}.get()

println "Fetched ${allUsers.size()} users concurrently"
```

116.2. Async with Error Handling

```
def client = new GroovyHttpClient("https://api.example.com")

def userIds = [1, 2, 3, 999, 5]  // 999 doesn't exist

def futures = userIds.collect { id ->
    client.get("/users/${id}")
        .thenApply { response ->
            [success: true, data: response.json]
        }
        .exceptionally { error ->
            [success: false, error: error.message, userId: id]
        }
}

// Wait for all (including failures)
def results = CompletableFuture.allOf(*futures).thenApply {
    futures.collect { it.join() }
}.get()

// Process results
def successful = results.findAll { it.success }
def failed = results.findAll { !it.success }
```

```
println "Successful: ${successful.size()}"
println "Failed: ${failed.size()}"

failed.each { println "Failed user ${it.userId}: ${it.error}" }
```

116.3. Parallel Processing

```
def client = new GroovyHttpClient("https://api.example.com")

def processUserBatch(List<Integer> userIds) {
    def futures = userIds.collect { id ->
        client.get("/users/${id}")
            .thenApply { response -> response.json }
            .thenApply { userData ->
                // Process each user
                processUser(userData)
                return userData
            }
    }

    // Wait for batch to complete
    return CompletableFuture.allOf(*futures)
        .thenApply { futures.collect { it.join() } }
        .get()
}

// Process in batches of 100
def allUserIds = (1..1000).toList()
def batchSize = 100

allUserIds.collate(batchSize).each { batch ->
    def processed = processUserBatch(batch)
    println "Processed batch of ${processed.size()} users"
}
```

Chapter 117. Testing Examples

117.1. Testing with HttpClientTestDsl

```
import org.softwood.http.HttpClientTestDsl
import spock.lang.Specification

class UserServiceSpec extends Specification {

    HttpClientTestDsl testDsl
    UserService service

    def setup() {
        testDsl = HttpClientTestDsl.create()
        testDsl.start()
        service = new UserService(testDsl.baseUrl)
    }

    def cleanup() {
        testDsl.stop()
    }

    def "should fetch user by ID"() {
        given: "a mock user endpoint"
        testDsl.when("/users/123").respondWith {
            status 200
            json([
                id: 123,
                name: "Alice Smith",
                email: "alice@example.com"
            ])
        }

        when: "fetching user"
        def user = service.getUser(123)

        then: "user data is correct"
        user.id == 123
        user.name == "Alice Smith"

        and: "request was made with correct headers"
        testDsl.verify("/users/123") {
            wasCalled(1)
            withMethod("GET")
            withHeader("Authorization")
        }
    }

    def "should handle user not found"() {
```

```
        given: "a 404 response"
        testDsl.when("/users/999").respondWith {
            status 404
            json([error: "User not found"])
        }

        when: "fetching non-existent user"
        service.getUser(999)

        then: "exception is thrown"
        thrown(UserNotFoundException)
    }
}
```

117.2. Testing Multipart Uploads

```
import org.softwood.http.HttpClientTestDsl
import spock.lang.Specification

class DocumentServiceSpec extends Specification {

    HttpClientTestDsl testDsl
    DocumentService service

    def setup() {
        testDsl = HttpClientTestDsl.create()
        testDsl.start()
        service = new DocumentService(testDsl.baseUrl)
    }

    def cleanup() {
        testDsl.stop()
    }

    def "should upload document"() {
        given: "a mock upload endpoint"
        testDsl.when("/documents").respondWith {
            status 201
            json([
                document_id: "doc_123",
                filename: "report.pdf",
                size: 12345
            ])
        }

        and: "a document to upload"
        def file = new File("test-data/report.pdf")

        when: "uploading document"
```

```
        def result = service.uploadDocument(
            file,
            "Monthly Report",
            "reports"
        )

        then: "upload succeeds"
        result.document_id == "doc_123"

        and: "request was made correctly"
        testDsl.verify("/documents") {
            wasCalled(1)
            withMethod("POST")
            withHeader("Content-Type")  // multipart/form-data
            withHeader("Authorization")
        }
    }
}
```

Chapter 118. Production Patterns

118.1. Rate Limiting

```
import java.util.concurrent.Semaphore
import java.util.concurrent.TimeUnit

class RateLimitedClient {
    private final GroovyHttpClient client
    private final Semaphore semaphore
    private final long refillPeriodMs

    RateLimitedClient(String baseUrl, int maxConcurrent, long refillPeriodMs) {
        this.client = new GroovyHttpClient(baseUrl)
        this.semaphore = new Semaphore(maxConcurrent)
        this.refillPeriodMs = refillPeriodMs

        // Background thread to refill permits
        Thread.startVirtualThread {
            while (true) {
                Thread.sleep(refillPeriodMs)
                semaphore.release(maxConcurrent - semaphore.availablePermits())
            }
        }
    }

    def get(String path) {
        semaphore.acquire()
        try {
            return client.getSync(path)
        } finally {
            // Don't release immediately - rate limit enforced by background thread
        }
    }
}

// Usage: Max 10 requests per second
def rateLimited = new RateLimitedClient(
    "https://api.example.com",
    10,      // Max concurrent
    1000     // Refill every 1 second
)

100.times { i ->
    def response = rateLimited.get("/data/${i}")
    println "Request ${i}: ${response.statusCode}"
}
```

=# Logging and Metrics

```
import java.util.concurrent.atomic.AtomicInteger
import java.util.concurrent.atomic.AtomicLong

class InstrumentedClient {
    private final GroovyHttpClient client
    private final AtomicInteger requestCount = new AtomicInteger(0)
    private final AtomicInteger errorCount = new AtomicInteger(0)
    private final AtomicLong totalLatency = new AtomicLong(0)

    InstrumentedClient(String baseUrl) {
        this.client = new GroovyHttpClient(baseUrl)

        // Add logging interceptor
        client.addInterceptor { ctx, next ->
            def requestId = UUID.randomUUID().toString()
            def start = System.currentTimeMillis()

            println "[${requestId}] → ${ctx.method} ${ctx.uri}"
            requestCount.incrementAndGet()

            next.call(ctx).whenComplete { response, error ->
                def elapsed = System.currentTimeMillis() - start
                totalLatency.addAndGet(elapsed)

                if (error) {
                    errorCount.incrementAndGet()
                    println "[${requestId}] □ Failed after ${elapsed}ms:
${error.message}"
                } else {
                    println "[${requestId}] ✓ ${response.statusCode} in ${elapsed}ms"
                }
            }
        }
    }

    def getMetrics() {
        def count = requestCount.get()
        def errors = errorCount.get()
        def avgLatency = count > 0 ? totalLatency.get() / count : 0

        return [
            requests: count,
            errors: errors,
            success_rate: count > 0 ? ((count - errors) / count * 100) : 0,
            avg_latency_ms: avgLatency
        ]
    }
```

```
    // Delegate methods
    def getSync(String path, Closure config = null) {
        client.getSync(path, config)
    }
}

// Usage
def instrumented = new InstrumentedClient("https://api.example.com")

// Make requests
100.times { instrumented.getSync("/data") }

// View metrics
def metrics = instrumented.getMetrics()
println "Requests: ${metrics.requests}"
println "Errors: ${metrics.errors}"
println "Success rate: ${metrics.success_rate}%"
println "Avg latency: ${metrics.avg_latency_ms}ms"
```

118.2. Configuration Management

```
import org.softwood.http.SecurityConfig
import org.softwood.http.GroovyHttpClient

class ApiClientFactory {

    static GroovyHttpClient createForEnvironment(String env) {
        switch (env.toLowerCase()) {
            case 'development':
                return createDevelopmentClient()
            case 'staging':
                return createStagingClient()
            case 'production':
                return createProductionClient()
            default:
                throw new IllegalArgumentException("Unknown environment: ${env}")
        }
    }

    private static GroovyHttpClient createDevelopmentClient() {
        def config = SecurityConfig.testing("http://localhost:8080")
            .withOverrides([
                requestTimeout: Duration.ofSeconds(60),
                failureThreshold: 100,  // Very tolerant
                enableLogging: true
            ])
        new GroovyHttpClient(config)
    }
```

```
    private static GroovyHttpClient createStagingClient() {
        def config = SecurityConfig.production("https://staging-api.example.com")
            .withOverrides([
                requestTimeout: Duration.ofSeconds(30),
                failureThreshold: 10,
                resetTimeoutMs: 30_000
            ])
        new GroovyHttpClient(config)
    }

    private static GroovyHttpClient createProductionClient() {
        def retryPolicy = new RetryPolicy.Config().tap {
            maxAttempts(3)
            backoffExponential(200, 2000, true)
            on5xx(true)
        }.build()

        def config = SecurityConfig.production("https://api.example.com")
            .withOverrides([
                requestTimeout: Duration.ofSeconds(10),
                connectTimeout: Duration.ofSeconds(5),
                failureThreshold: 5,
                resetTimeoutMs: 30_000,
                retryPolicy: retryPolicy,
                enforceHostnameVerification: true,
                allowedTlsProtocols: ["TLSv1.3"],
                enableMetrics: true
            ])
        new GroovyHttpClient(config)
    }
}

// Usage
def env = System.getenv("APP_ENV") ?: "development"
def client = ApiClientFactory.createForEnvironment(env)
```

Chapter 119. Additional Examples in Test Suite

The project includes comprehensive test suites demonstrating advanced usage:

119.1. Client Tests

Located in `src/test/groovy/org/softwood/http/clientTests/`:

`GroovyHttpClientSpec.groovy`	Basic HTTP operations, headers, cookies, timeouts
`GroovyHttpClientMultipartSpec.groovy`	Multipart uploads (DSL and API), file downloads
`GroovyHttpClientHttp2Spec.groovy`	HTTP/2 features, concurrent requests
`CircuitBreakerSpec.groovy`	Circuit breaker states, failure thresholds, recovery

119.2. Testing Framework Tests

Located in `src/test/groovy/org/softwood/http/testingFrameworkTests/`:

`HttpClientTestDslSpec.groovy`	Mock endpoints, request verification, response building
`HttpClientTestTraitSpec.groovy`	Spock trait integration, simplified testing

119.3. Running the Tests

```
# Run all tests
./gradlew test

# Run specific test class
./gradlew test --tests GroovyHttpClientSpec

# Run tests with detailed output
./gradlew test --info

# Generate test report
./gradlew test
# Open build/reports/tests/test/index.html
```

119.4. Test Report

After running tests, open the HTML report to see:

- Detailed test results
- Execution time per test

- Pass/fail status
- Code coverage (if configured)

The test suite serves as living documentation - when in doubt, check the tests!

Chapter 120. Next Steps

- Review the API Reference for complete method signatures
- Check Troubleshooting for common issues
- Explore the test suite for more examples
- Contribute your own examples via GitHub pull requests



Have a unique use case? Share it with the community! Submit a pull request adding your example to this chapter.

Appendix A: API Reference

...

Appendix B: Troubleshooting

...